

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Политехнический институт
Факультет приборостроения,
информационных технологий и
электроники

Кафедра
«Информационная безопасность
систем и технологий»

Специальность 100503 –

Специализация -

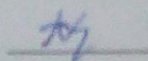
Информационная безопасность
автоматизированных систем
Защищенные автоматизированные
системы управления

ДИПЛОМНЫЙ ПРОЕКТ

на тему

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ДЛЯ ПОСТРОЕНИЯ КРИПТОСТОЙКОГО
НЕЛИНЕЙНОГО БИЕКТИВНОГО ПРЕОБРАЗОВАНИЯ

Студент



Старкин Алексей Валерьевич

Руководитель



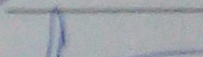
Сомкин С.А.

Нормоконтролёр



Иванов А.П.

Рецензент Начальник НИС-103



Акмасев А.Ж.

Работа допущена к защите (протокол заседания кафедры от 20.06.17 № 14)

Заведующий кафедрой



С.Л. Зефирова

Работа защищена с отметкой Одн.м.т.о. (протокол заседания ГЭК
от 25.06.17 № 14)

Секретарь ГЭК



А.П. Иванов

Пенза 2017

РЕФЕРАТ

Пояснительная записка содержит 107 с., 1 прил., 10 табл., 40 рис., 23 источника.

ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ, АЛГОРИТМ ШИФРОВАНИЯ, КРИПТОГРАФИЧЕСКАЯ СТОЙКОСТЬ, НЕЛИНЕЙНОЕ БИЕКТИВНОЕ ПРЕОБРАЗОВАНИЕ, БУЛЕВА ФУНКЦИЯ

Объектом исследования дипломного проектирования является программное обеспечение, предназначенное для построения криптостойкого нелинейного биективного преобразования.

Целями дипломного проекта являются определение криптографических требований, предъявляемых к криптостойкому нелинейному биективному преобразованию, и разработка программного обеспечения для построения криптостойкого нелинейного биективного преобразования.

В процессе выполнения дипломного проекта был проведен обзор применения нелинейного биективного преобразования в криптографических алгоритмах; определены характеристики, необходимые для обеспечения криптостойкости нелинейного биективного преобразования, а также разработан алгоритм генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности и методика и рекомендации по работе с разработанным программным обеспечением.

В результате выполнения дипломного проекта разработано программное обеспечение, которое осуществляет работу в следующих режимах:

- проверка выбранного нелинейного биективного преобразования на соответствие заданным требованиям;
- поиск подмножества нелинейных биективных преобразований, соответствующих заданным требованиям;

Генерация узлов замены осуществлена в соответствии с требованиями «доказуемой» псевдослучайности. В качестве базового криптографического механизма использован ГОСТ Р 34.12-2015 («Кузнечик») в режиме

гаммирования (ГОСТ Р 34.13-2015). Разработанное программное обеспечение имеет графический интерфейс и функционирует на ОС семейства Windows версии не ниже Windows 7.

СОДЕРЖАНИЕ

Введение.....	9
1 Обзор применения нелинейного биективного преобразования в криптографических алгоритмах	11
2 Определение списка требований, необходимых для обеспечения криптостойкости нелинейного биективного преобразования	18
2.1 Общие требования к криптостойкости нелинейного биективного преобразования.....	18
2.2 Матрица встречаемости разностей переходов ненулевых биграмм	18
2.3 Нелинейность булевых функций	20
2.4 Алгебраическая степень булевых функций.....	23
2.5 Разностная характеристика булевых функций.....	25
2.6 Цикловая структура подстановок.....	26
2.7 Глобальный лавинный критерий	27
2.8 Определение границ требований, необходимых для обеспечения криптостойкости нелинейного биективного преобразования	29
3 Разработка алгоритма генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности.....	41
4 Разработка методики и рекомендации по работе с программным обеспечением	46
5 Результаты проведения исследования по поиску S-блоков, удовлетворяющих требованиям, и проверке существующих S-блоков.....	53
5.1 Поиск S-блоков, удовлетворяющих криптографическим требованиям	53
5.2 Проверка применяемых в криптографических алгоритмах S-блоков.....	59
Заключение	74
Список использованных источников	76

Приложение А Исходный код программного обеспечения для построения криптостойкого нелинейного биективного преобразования.....	79
Приложение Б Акт о внедрении результатов выпускной квалификационной работы.....	106

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяются следующие термины с соответствующими определениями:

— информационная безопасность – состояние защищенности личности, общества и государства от внутренних и внешних информационных угроз, при котором обеспечиваются реализация конституционных прав и свобод человека и гражданина, достойные качество и уровень жизни граждан, суверенитет, территориальная целостность и устойчивое социально-экономическое-развитие Российской Федерации, оборона и безопасность государства; [1]

— блочный шифр – шифр из класса симметричных криптографических методов, в котором алгоритм зашифрования применяется к блокам открытого текста для получения блоков шифртекста; [2]

— алгоритм зашифрования – алгоритм, реализующий зашифрование, т.е. преобразующий открытый текст в шифртекст; [2]

— ключ – изменяемый параметр в виде последовательности символов, определяющий криптографическое преобразование; [2]

— итерационный ключ – последовательность символов, вычисляемая в процессе развертывания ключа шифра, и определяющая преобразование на одной итерации блочного шифра; [2]

— криптографическое преобразование – преобразование данных при помощи шифрования и (или выработки) имитовставки; [3]

— гаммирование – процесс наложения по определенному закону гаммы шифра на открытые данные; [3]

— гамма шифра – псевдослучайная двоичная последовательность, вырабатываемая по заданному алгоритму для зашифрования открытых данных и расшифрования зашифрованных данных; [3]

- зашифрование данных – процесс преобразования открытых данных в зашифрованные при помощи шифра; [3]
- криптографическая защита - защита данных при помощи криптографического преобразования данных; [3]
- синхропосылка – комбинация знаков, передаваемая по каналу связи и предназначенная для инициализации алгоритма шифрования; [4]
- счетчик-строка бит длины, равной длине блока блочного шифра, используемая при шифровании в режиме гаммирования; [4]
- криптографическая стойкость - способность криптографического алгоритма противостоять криптоанализу. [5]

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчете применяются следующие обозначения и сокращения:

ИБ – информационная безопасность;

ПО – программное обеспечение;

ЭВМ – электронно-вычислительная машина;

$P_2(n)$ - множество всех булевых функций от n переменных,

V_n -множество двоичных векторов длины n ;

(x, y) – скалярное произведение двоичных векторов;

$\Delta_f(u)$ -автокорреляционная функция;

$GF(q)$ – конечное поле из q элементов.

ВВЕДЕНИЕ

В настоящее время информационная безопасность обретает все большее значение во всех сферах жизни общества. Особенно остро встает вопрос защиты информации, составляющей коммерческую и государственную тайны, в организациях различных форм собственности, поскольку в случае ее компрометации владельцы активов имеют риск получения ущерба как материального, так и репутационного.

Одним из способов обеспечения конфиденциальности и целостности защищаемой информации являются криптографические методы защиты информации: зашифрование информации и/или вычисление имитовставки, вычисление криптографической функции хеширования.

Существуют следующие виды криптографических алгоритмов: блочные, зашифровывающие блоки текстов фиксированной длины, и поточные, которые осуществляют зашифрование путем наложения вырабатываемой гаммы на открытый текст.

Существуют два основных вида построения блочных криптографических алгоритмов: сеть Фейстеля и подстановочно-перестановочная сеть (SP-сеть).

Вне зависимости от типа используемой сети в данных криптографических алгоритмах используется нелинейное биективное преобразование, которое представлено в виде подстановок (S-блоков).

Оно используется для перемешивания битов данных. Согласно [6] перемешивание понимается как свойство шифрующего преобразования усложнять взаимосвязи между элементами данных, что затрудняет восстановление функциональных и статистических связей между открытым текстом, ключом и зашифрованным текстом. Данное преобразование является биективным, поскольку каждому элементу одного множества соответствует ровно один элемент другого множества, при этом существует обратное отображение, которое обладает тем же свойством.

Целями данного дипломного проекта являются определение криптографических требований, предъявляемых к криптостойкому нелинейному биективному преобразованию и разработка программного обеспечения для построения криптостойкого нелинейного биективного преобразования.

На этапах дипломного проекта был проведен обзор применения нелинейного биективного преобразования в криптографических алгоритмах; определены требования, необходимые для обеспечения криптостойкости нелинейного биективного преобразования; разработан алгоритм генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности и методика и рекомендации по работе с разработанным программным обеспечением.

1 Обзор применения нелинейного биективного преобразования в криптографических алгоритмах

В настоящее время в блочных и поточных криптографических алгоритмах шифрования активно применяется нелинейное биективное преобразование. Оно используется для перемешивания битов данных. Согласно [6] перемешивание – это свойство шифрующего преобразования, позволяющее усложнять взаимосвязи между элементами данных, что затрудняет восстановление функциональных и статистических связей между открытым текстом, ключом и зашифрованным текстом. Данное преобразование является биективным, поскольку при таком отображении каждому элементу одного множества соответствует ровно один элемент другого множества той же мощности, при этом существует обратное отображение, которое обладает тем же свойством.

В российском стандарте ГОСТ Р 34.12-2015 представлены два блочных криптографических алгоритма, использующих различные криптографические схемы:

- алгоритм блочного шифрования с длиной блока 128 бит и длиной ключа 256 бит (шифр «Кузнечик»), использующий подстановочно-перестановочную сеть;
- алгоритм блочного шифрования с длиной блока 64 бита и длиной ключа 256 бит (шифр «Магма»), использующий сеть Фейстеля.

Принцип использования сети Фейстеля заключается в следующем [7]: берется блок длиной n и разделяется на две половины длиной $n/2$: L и R . При этом длина блока является четным числом. Можно определить итеративный блочный шифр, в котором результат i -ой итерации определяется результатом предыдущего этапа:

$$\begin{aligned} L_i &= R_{i-1} \oplus f(L_{i-1}, K_{i-1}); \\ R_i &= L_{i-1}, \end{aligned} \tag{1}$$

где K_i - это итерационный ключ, используемый на i -ой итерации, функция f - произвольная функция итерации.

Схема сети Фейстеля представлена на рисунке 1.

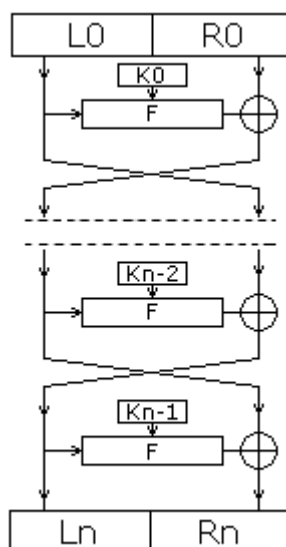


Рисунок 1 – Схема Фейстеля

В алгоритме блочного шифрования «Магма», использующем сеть Фейстеля, функция f представлена в виде, изображенном на рисунке 2.

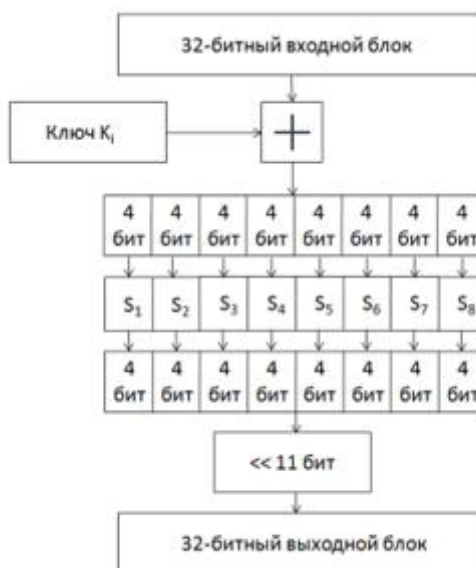


Рисунок 2 - Функция f , используемая в алгоритме блочного шифрования «Магма»

В функции f 32-битный входной блок складывается по модулю 2^{32} с итерационным ключом K_i . Далее полученное значение разбивается на блоки по 4 бита, которые проходят через S-блоки, а происходит сдвиг на 11 бит влево.

Алгоритм шифрования, использующий схему Фейстеля, обратим, если можно восстановить исходные данные функции f на каждом этапе. Сама функция f может являться настолько сложной, насколько необходимо, поскольку структура сети Фейстеля позволяет использовать функцию f в операции и зашифрования, и расшифрования.

Подстановочно-перестановочная сеть (SP-сеть) представляет собой несколько повторяющихся итераций, в каждой из которых поочередно происходит нелинейное (S-блок) и линейное (P-блок) преобразования. [7]

Перед выполнением каждой итерации происходит наложение итерационного ключа на обрабатываемое сообщение.

Схема подстановочно-перестановочной сети на примере трех раундов представлена на рисунке 3.

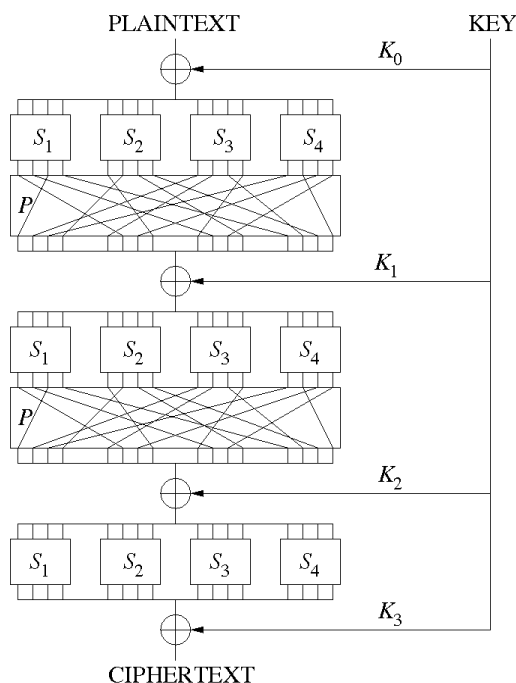


Рисунок 3 – Пример подстановочно-перестановочной сети, состоящей из трех раундов

В алгоритме блочного шифрования «Кузнечик» используется подстановочно-перестановочная сеть, схема которой представлена на рисунке 4.

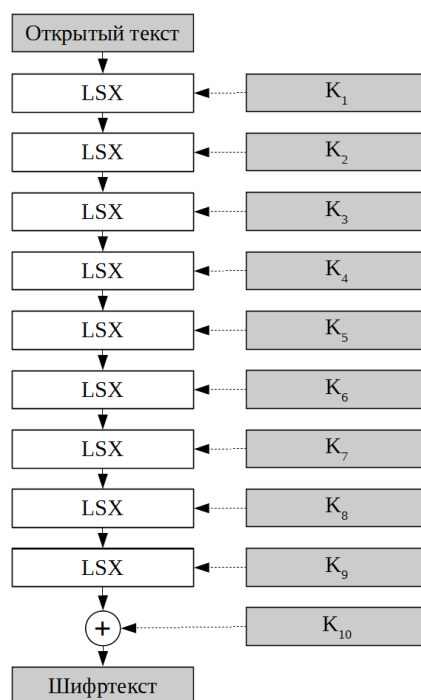


Рисунок 4 - Схема работы алгоритма блочного шифрования «Кузнечик»

В схеме, представленном на рисунке 3, X – сложение по модулю 2 итерационного ключа и текста; S - нелинейное биективное преобразование; L - линейное биективное преобразование.

В зарубежных алгоритмах шифрования нелинейное биективное преобразование используется в таких алгоритмах шифрования, как DES, AES, Blowfish и т.д.

Таким образом, на примере алгоритмов блочного шифрования российского стандарта ГОСТ Р 34.12-2015 показано, что вне зависимости от используемой криптографической схемы нелинейное биективное преобразование широко используется с целью усложнения криптоанализа.

В современных блочных и поточных шифрах данное преобразование представляется в виде S-блоков - таблиц подстановки π определенной размерности в зависимости от используемого криптографического алгоритма. К примеру, в российском стандарте ГОСТ Р 34.12-2015 в шифре «Кузнечик»

применяется S-блок в виде подстановки степени 256, а в шифре «Магма» – 8 S-блоков степени 16 каждый. [2] Примеры представлены на рисунках 5 и 6.

$\pi' = (252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4, 77, 233, 119, 240, 219, 147, 46, 153, 186, 23, 54, 241, 187, 20, 205, 95, 193, 249, 24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66, 139, 1, 142, 79, 5, 132, 2, 174, 227, 106, 143, 160, 6, 11, 237, 152, 127, 212, 211, 31, 235, 52, 44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253, 58, 206, 204, 181, 112, 14, 86, 8, 12, 118, 18, 191, 114, 19, 71, 156, 183, 93, 135, 21, 161, 150, 41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158, 178, 177, 50, 117, 25, 61, 255, 53, 138, 126, 109, 84, 198, 128, 195, 189, 13, 87, 223, 245, 36, 169, 62, 168, 67, 201, 215, 121, 214, 246, 124, 34, 185, 3, 224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78, 51, 10, 74, 167, 151, 96, 115, 30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65, 173, 69, 70, 146, 39, 94, 85, 47, 140, 163, 165, 125, 105, 213, 149, 59, 7, 88, 179, 64, 134, 172, 29, 247, 48, 55, 107, 228, 136, 217, 231, 137, 225, 27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144, 202, 216, 133, 97, 32, 113, 103, 164, 45, 43, 9, 91, 203, 155, 37, 208, 190, 229, 108, 82, 89, 166, 116, 210, 230, 244, 180, 192, 209, 102, 175, 194, 57, 75, 99, 182).$

Рисунок 5 – Нелинейное биективное преобразование, представленное в алгоритме блочного шифрования «Кузнечик»

$\pi_0' = (12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1);$
 $\pi_1' = (6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15);$
 $\pi_2' = (11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0);$
 $\pi_3' = (12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11);$
 $\pi_4' = (7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12);$
 $\pi_5' = (5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0);$
 $\pi_6' = (8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7);$
 $\pi_7' = (1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2).$

Рисунок 6 – Нелинейное биективное преобразование, представленное в алгоритме блочного шифрования «Магма»

Алгоритм DES (Data Encryption Standard), принятый в США в 1977 году, так же, как и алгоритм «Магма», построен на сети Фейстеля. [8]

Нелинейное биективное преобразование, используемое в данном криптографическом алгоритме, представлено на рисунке 7.

14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7	
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8	S_1
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0	
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13	
15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10	
3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5	S_2
0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15	
13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9	
10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8	
13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1	S_3
13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7	
1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12	
7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15	
13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9	S_4
10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4	
3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14	
2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9	
14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6	S_5
4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14	
11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3	
12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11	
10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8	S_6
9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6	
4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13	
4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1	
13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6	S_7
1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2	
6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12	
13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7	
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2	S_8
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8	
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11	

Рисунок 7 – Нелинейное биективное преобразование, используемое в алгоритме блочного шифрования DES

Алгоритм AES (Advanced Encryption Standart), принятый в США в 2000 году, так же, как и алгоритм «Кузнечик», построен на подстановочно-перестановочной сети. [9]

Нелинейное биективное преобразование, используемое в данном криптографическом алгоритме, представлен на рисунке 8.

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Рисунок 8 – Нелинейное биективное преобразование, используемое в алгоритме AES

Использование S-блоков в совокупности с линейным преобразованием (преобразование P) в раундовых функциях криптографических алгоритмов позволяет добиться свойства «лавинного эффекта», т.е. при изменении малого числа бит либо открытого текста, либо ключа должно изменяться большое число бит зашифрованного текста.

Следовательно, необходимо, чтобы в криптографических алгоритмах применялись S-блоки, удовлетворяющие некоторым криптографическим требованиям с целью усложнения проведения противником криптографического анализа.

2 Определение списка требований, необходимых для обеспечения криптостойкости нелинейного биективного преобразования

2.1 Общие требования к криптостойкости нелинейного биективного преобразования

Нелинейное биективное преобразование, представленное в виде S-блоков, должно обеспечивать затруднение противнику восстановления функциональных и статистических связей между открытым текстом, ключом и зашифрованным текстом.

Для обеспечения такого свойства S-блоки должны удовлетворять некоторым криптографическим требованиям, при этом необходимо соблюдать компромисс в наборе данных характеристик и их определяемых границах, поскольку применяемые в криптографических алгоритмах S-блоки должны одновременно удовлетворять:

- критерию лавинного эффекта;
- требованиям по защите используемых криптографических алгоритмов от линейного криптоанализа;
- требованиям по защите используемых криптографических алгоритмов от дифференциального криптоанализа;
- требованиям по защите используемых криптографических алгоритмов от алгебраической атаки.

Далее будут рассматриваться S-блоки $\pi(x)$ степени 16 и 256, поскольку S-блоки данной размерности чаще всего используются в криптографических алгоритмах.

2.2 Матрица встречаемости разностей переходов ненулевых биграмм

Согласно [10] для каждого S-блока степени $N \in \overline{16, 256}$ существует матрица встречаемости разностей переходов ненулевых биграмм $P(\pi)$ размера $(N-1) \times (N-1)$, где p_{ij} – число решений системы уравнений:

$$\begin{cases} x_2 - x_1 = i, \\ \pi(x_2) - \pi(x_1) = j \end{cases}, \text{ где } i, j \neq 0. \quad (2)$$

Из формулы (2) вытекает, что

$$\pi(i + x_1) - \pi(x_1) = j, \text{ где } i, j \neq 0. \quad (3)$$

Из формулы (3) следует, что сумма элементов p_{ij} по каждой строке i и каждому столбцу j матрицы $P(\pi)$ равна 2^N . Таким образом, в идеальном случае, если бы в данной матрице отсутствовали нулевые значения, то тогда все элементы матрицы p_{ij} в i -ой строке, были равны 1, за исключением одного, который равен 2.

Таким образом, матрица встречаемости разностей переходов ненулевых биграмм $P(\pi)$ позволяет определить степень «лавиного эффекта»: данное свойство будет тем лучше, чем меньше нулей будет в матрице $P(\pi)$. Также необходимо, чтобы распределение значений строк данной матрицы было близко к равномерному, т.е. ее максимальный элемент должен быть как можно меньше.

Существуют подстановки, называемыми логарифмическими (т.е. значения подстановок определены через логарифм), где наиболее ярко выражен «лавиный эффект», так как именно в таких подстановках в матрице $P(\pi)$ определено минимально возможное число нулей в матрице частот.

Пример логарифмической подстановки степени 16 представлен на рисунке 9. [10]

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\pi(x)$	14	12	3	7	9	15	8	13	0	6	2	10	5	4	1	11

Рисунок 9 – Логарифмическая подстановка степени 16

Матрица встречаемости разностей переходов ненулевых биграмм данной логарифмической подстановки, представлена на рисунке 10. [10]

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	0	1	2	1	1	2	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	1	2	1	1	1	1	1	1	1
3	2	1	0	1	1	1	1	1	1	2	1	1	1	1	1
4	1	1	1	0	2	1	2	1	1	1	1	1	1	1	1
5	1	1	1	2	0	1	1	1	2	1	1	1	1	1	1
6	2	1	1	1	1	0	1	1	1	1	1	1	2	1	1
7	1	1	1	2	1	1	0	1	1	1	2	1	1	1	1
8	1	2	1	1	1	1	1	0	1	1	1	1	1	2	1
9	1	1	1	1	2	1	1	1	0	1	1	2	1	1	1
10	1	1	2	1	1	1	1	1	1	0	1	1	1	1	2
11	1	1	1	1	1	1	2	1	1	1	0	2	1	1	1
12	1	1	1	1	1	1	1	1	2	1	2	0	1	1	1
13	1	1	1	1	1	2	1	1	1	1	1	1	0	1	2
14	1	1	1	1	1	1	1	2	1	1	1	1	1	1	1
15	1	1	1	1	1	1	1	1	1	2	1	1	2	1	0

Рисунок 10 – Матрица встречаемости разностей переходов ненулевых биграмм

Данная подстановка имеет минимально возможное количество нулей в матрице встречаемости разностей переходов ненулевых биграмм.

Логарифмические подстановки, несмотря на отличные характеристики в данной области, в настоящий момент не применяются, поскольку их общее количество настолько мало, что их можно определить методом полного перебора.

Самыми плохими подстановками являются линейные подстановки вида $\pi(x) = ax + b$, поскольку число нулей в $P(\pi)$ будет максимальным.

Таким образом, определены следующие требования к S-блокам:

- максимальный элемент матрицы встречаемости разностей переходов ненулевых биграмм $P(\pi) \max_{i,j \neq 0} p_{ij} \leq P_1$;
- число ненулевых элементов матрицы $P(\pi)$.

2.3 Нелинейность булевых функций

Данная характеристика призвана затруднить противнику проведение линейного криптоанализа.

Данный вид криптоанализа был предложен японским криптографом Мицуру Мацуи в 1993 году. Основной смысл данной атаки – нахождение линейного статистического аналога, выполняющейся с вероятностью, намного отличной от 50 %, и определение с его помощью секретного ключа.

Линейным статистическим аналогом является соотношение вида

$$(\alpha, P) \oplus (\beta, C) = (\gamma, K), \quad (4)$$

где α, β, γ - некоторые двоичные векторы, P, C, K - двоичные векторы открытого текста, зашифрованного текста и секретного ключа соответственно. Применяются для осуществления данного вида криптоанализа те линейные статистические аналоги, вероятность которых заметно отличается от 0.5 в меньшую или большую сторону. Вероятность определяется путем определения равенства между левой и правой частью формулы (4). [11, 12]

После нахождения линейного статистического аналога, максимально отличающегося от 0.5, производится выборка открытого и зашифрованного текста и с его помощью определяется наиболее вероятный набор значений битов ключа.

При описании данной характеристики руководствуются тем, любой S-блок степени 2^n можно представить в виде совокупности координатных булевых функций $f_i(x_1, x_2, \dots, x_n), i \in \overline{1, n}$.

Одним из методов защиты от линейного криптоанализа при создании S-блоков является высокая степень нелинейности координатных функций. [13, 14]

Аффинной функцией является функция вида

$$g(x_1, \dots, x_n) = a_1 x_1 \oplus a_2 x_2 \oplus \dots \oplus a_n x_n \oplus b, \quad (5)$$

где $a_i, b \in GF(2), i \in \overline{1, n}$. Если коэффициент $b = 0$, то такая функция является линейной.

Согласно [15] степень нелинейности булевых функций N_f определяется как минимальное расстояние Хэмминга между i -ой булевой функцией и линейными функциями.

Степень нелинейности N_f определяется при помощи преобразования Уолша-Адамара.

Преобразованием Уолша-Адамара функции $f \in P_2(n)$, где $P_2(n)$ - множество всех булевых функций от n переменных, называется функция $\hat{f}: V_n \rightarrow Z$, где для каждого $a \in V_n$ [16]:

$$\hat{f}(a) = \sum_{x \in V_n} (-1)^{f(x) \oplus (a, x)} \quad (6)$$

Значения $\hat{f}(a)$ называются коэффициентами преобразования Уолша-Адамара.

Степень нелинейности булевых функций N_f , где $f \in P_2(n)$ выражается S-блок, определяется как:

$$N_f = 2^{n-1} - \frac{1}{2} \max_{a \in V_n} |\hat{f}(a)|. \quad (7)$$

Максимальной степенью нелинейности обладают булевы функции, называемые бент-функциями.

Для бент-функции значение $N_f = 2^{n-1} - 2^{n/2-1}$. [16]

Определение степени нелинейности S-блоков основывается на вычислении минимального значения нелинейности координатных функций, описывающих данные S-блоки. Соответственно, чем ближе минимальная степень нелинейности к максимально возможному, тем лучше свойства S-блоков с точки зрения противостояния линейному криптоанализу.

Таким образом определено еще одно требование к S-блокам: степень нелинейности N_f , верхняя оценка которой равна $N_f \leq 2^{n-1} - 2^{n/2-1}$.

2.4 Алгебраическая степень булевых функций

Данная характеристика направлена на усложнение проведения противником алгебраической атаки, смысл которой состоит в построении системы квадратичных уравнений и нахождении секретного ключа при решении данной системы при помощи метода Гаусса. [17]

При описании данной характеристики руководствуются тем, что любой S-блок степени 2^n можно представить в виде совокупности координатных булевых функций $f_i(x_1, x_2, \dots, x_n), i \in \overline{1, n}$.

Алгебраической степенью булевой функции f называется наибольшая степень монома алгебраической нормальной формы (полинома Жегалкина) булевой функции. В свою очередь степенью монома называется количество переменных в одном слагаемом полинома Жегалкина булевой функции. [13]

Полиномом Жегалкина называется многочлен над двоичным полем F_2 вида

$$P(x_1, x_2, \dots, x_n) = a \oplus a_1 x_1 \oplus a_2 x_2 \oplus \dots \oplus a_n x_n \oplus a_{12} x_1 x_2 \oplus a_{13} x_1 x_3 \oplus \dots \oplus a_{1\dots n} x_1 \dots x_n, (8)$$

где $a, \dots, a_{1\dots n} \in \{0, 1\}$.

Полином Жегалкина может строиться несколькими способами [18]:

- с помощью эквивалентных преобразований ДНФ;
- с помощью эквивалентных преобразований СДНФ;
- с применением карты Карно;
- с помощью метода треугольника;
- с помощью метода Паскаля;
- с помощью метода суммирования.

В настоящем дипломном проекте для построения полинома Жегалкина использован метод треугольника, который заключается в следующем [18]:

- строится полная таблица истинности булевой функции;

- строится вспомогательная треугольная таблица, в которой первый столбец совпадает со столбцом значений функции в таблице истинности;
- в каждом последующем столбце ячейка получается путем сложения по модулю 2 двух ячеек предыдущего столбца – стоящей в той же строке и строкой ниже;
- столбцы вспомогательной таблицы нумеруются двоичными кодами том же порядке, что и строки таблицы истинности;
- каждому двоичному коду ставится в соответствие один из членов полинома Жегалкина в зависимости от позиции кода, в которых стоят единицы;
- если в верхней строке какого-либо столбца стоит единица, то соответствующий член присутствует в полиноме Жегалкина.

Пример построения полинома Жегалкина булевой функции представлен на рисунке 11.

<i>A</i>	<i>B</i>	<i>C</i>	<i>P</i>	000	001	010	011	100	101	110	111
0	0	0	1	1	<i>C</i>	<i>B</i>	<i>BC</i>	<i>A</i>	<i>AC</i>	<i>AB</i>	<i>ABC</i>
0	0	1	0	1	1	0	0	0	0	1	0
0	1	0	1	0	1	0	0	1	0		
0	1	1	0	0	1	0	1	1			
1	0	0	1	1	1	1	0				
1	0	1	0	0	0	1					
1	1	0	0	0	1						
1	1	1	1	1							

$$P = 1 \oplus C \oplus AB$$

Рисунок 11 – Пример построения полинома Жегалкина булевой функции

Как видно из рисунка 11, полиномом Жегалкина является выражение *P*.

Поиск S-блоков, стойких к алгебраической атаке, основан на определении минимальной алгебраической степени среди координатных функций.

Как определено в [19], для противодействия алгебраической атаке необходимо, чтобы минимальная алгебраическая степень из n булевых функций, которые описывают S-блок, должна быть как можно выше, т.к. чем больше эта степень, тем больше уравнений и переменных будет в системе квадратичных уравнений. Тем самым сложность проведения алгебраической атаки противником будет возрастать.

2.5 Разностная характеристика булевых функций

Данная характеристика направлена на усложнение проведения противником дифференциального криптоанализа.

Данный вид криптоанализа был описан Эли Бихамом и Ади Шамиром в 1990 году.

Замысел данной атаки для блочных шифров с k раундами заключается в поиске наиболее вероятного i -го дифференциала ($i \in \overline{1, k}$) с целью определения секретного ключа.

Дифференциалом i -ой итерации называется пара значений (x, y) , при которых существуют пара таких открытых текстов P и P' , что разность этих открытых текстов равна $P \oplus P' = x$, а разность соответствующих зашифрованных текстов C_i и C_i' равна $C_i \oplus C_i' = y$, где C_i и C_i' - зашифрованные тексты на i -ом раунде зашифрования.

Вероятностью i -го дифференциала является условная вероятность появления разности зашифрованных текстов $C_i \oplus C_i' = y$ при условии разности открытых текстов $P \oplus P' = x$, т.е. $P(C_i \oplus C_i' = y / P \oplus P' = x)$.

После определения наиболее вероятного дифференциала $(x, y)_i$ происходит выборка из значений P, P', C, C' таких, что $P \oplus P' = x$.

Далее, перебирая раундовые ключи K_{i+1} , расшифровываются зашифрованные тексты C и C' до i -го раунда, получая промежуточные значения на итерациях C_i и C_i' и проверяя условие выполнения выражения $C_i \oplus C_i' = y$. Тот ключ, при котором равенство выполняется наиболее часто, полагается за верный. [12, 13]

При описании данной характеристики руководствуются тем, любой S-блок степени 2^n можно представить в виде совокупности координатных булевых функций $f_i(x_1, x_2, \dots, x_n), i \in \overline{1, n}$.

Булева функция f является дифференциально δ -равномерной, если при любых $a \neq 0, b \in V_n$ уравнение $f(x) \oplus f(x \oplus a) = b$. [16]

Вычисление δ -равномерности булевой функции осуществляется на основе полного перебора $a \neq 0, b \in V_n$ и определении количества наборов $\{a, b\}$, при которых выполняется уравнение $f(x) \oplus f(x \oplus a) = b$.

Поиск S-блоков, стойких к дифференциальному криптоанализу, основан на определении максимальной дифференциальной характеристики среди координатных функций.

В [13, 20] определено, что для защиты от дифференциального криптоанализа булевые функции, которые описывают S-блок, значение δ должно быть как можно меньше, поскольку в таком случае распределение вероятностей дифференциалов будет близко к равномерному.

В [20] определена нижняя граница параметра $\delta \geq 2$. Булевые функции, значение δ -равномерности которых равно 2, называются почти совершенно нелинейными функциями (APN-функциями). [13]

2.6 Цикловая структура подстановок

Любой S-блок степени 2^n можно представить в виде произведений непересекающихся циклов.

Пример представления подстановки циклами показан на рисунке 12.

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 5 & 1 & 6 & 4 & 2 & 3 \end{pmatrix} = (1 \ 5 \ 2)(3 \ 6)(4)$$

Рисунок 12 – Пример представления подстановки циклами

При определении циклической структуры S-блока оперируют явлением возврата. Оно означает, что если S-блок использовать определенное число итераций, то любая исходная последовательность отобразится сама на себя. [21]

Это явление зависит от длины циклов. В [21] показано, что для S-блока чем больше минимальная длина цикла, тем лучшими криптографическими свойствами он обладает.

Также в [21] определено, что количество циклов должно быть как можно меньшим с целью укрупнения длин циклов. Фактически здесь определена зависимость между количеством циклов и их длинами, т.е. чем меньше количество циклов, тем больше будет их длина.

Особо в данной характеристике выделяются случаи, когда минимальная длина цикла составляет единицу – это неподвижная точка подстановки.

В случае наличия неподвижных точек, т.е. $\pi(x) = x$, фактически отсутствует операция перемешивания, которая является основным предназначением S-блоков в криптографических алгоритмах, тем самым уменьшается криптостойкость алгоритмов, в которых используются данные S-блоки.

Таким образом, можно выделить три основных свойства, касающиеся циклической структуры S-блоков:

- минимальная длина циклов, представляющих S-блок;
- количество циклов;
- отсутствие неподвижных точек S-блока.

2.7 Глобальный лавинный критерий

При описании данной характеристики руководствуются тем, что любой S-блок степени 2^n можно представить в виде совокупности координатных

булевых функций $f_i(x_1, x_2, \dots, x_n), i \in \overline{1, n}$.

В [22] был определен строгий лавинный критерий (SAC), который гласит, что если при $\forall i \in \overline{1, n}$ и при случайном выборе вектора $x \in V_n$ и замене i -ой компоненты в нем на противоположное значение булевой функции $f(x)$ изменится с вероятностью 0.5.

Или: функция f удовлетворяет строгому лавинному критерию, если $\Delta_f(u) = 0$ для таких u , у которых вес Хэмминга равен 1, где $\Delta_f(u)$ - автокорреляционная функция, которая равна:

$$\Delta_f(u) = \sum_{x \in V_2} (-1)^{f(x) \oplus f(x \oplus u)}. \quad (9)$$

Строгий лавинный критерий и его производные (критерий распространения и строгий лавинный критерий порядка m) отражают важное одноименное свойство в функциях, разрабатываемых для криптографических алгоритмов и алгоритмов хеширования.

Как было сказано ранее, чтобы функция удовлетворяла SAC, необходимо, чтобы автокорреляционная функция $\Delta_f(u) = 0$ для таких направлений u , у которых вес Хэмминга равен 1. Но при этом, одновременно удовлетворяя SAC, функция может иметь большое число векторов, у которых вес Хэмминга больше, чем 1 и которые являются линейными структурами (векторами α , при которых выражение $f(x) \oplus f(x \oplus \alpha)$ является константой). Применение таких функций может повлечь за собой потенциальный риск безопасности.

Поэтому в [23] предложен критерий, который оценивает обобщенные лавинные характеристики булевых функций – глобальный лавинный критерий (GAC). Данный критерий представлен следующими параметрами:

– сумма квадратов значений автокорреляционной функции по всем направлениям u :

$$\sigma_f = \sum_{u \in V_b} \Delta_f^2(u); \quad (10)$$

– максимальное значение автокорреляционной функции по всем направлениям u , кроме $u \neq 0$:

$$\Delta_f = \max_{u \in V_2, u \neq 0} \Delta_f(u) \quad (11)$$

Поиск S-блоков, обеспечивающих хорошие лавинные характеристики, основан на нахождении максимальных значений параметров глобального лавинного критерия среди координатных функций.

В [23] определено, что чем ближе значения σ_f и Δ_f к нулю, тем лучше лавинные характеристики булевой функции.

2.8 Определение границ требований, необходимых для обеспечения криптостойкости нелинейного биективного преобразования

После определения в п.п.2.1-2.6 требований, необходимых для обеспечения криптостойкости нелинейного биективного преобразования, применяемого в криптографических алгоритмах, установлены границы этих требований.

Данные границы определены при помощи использования статистических характеристик требований.

Так как мощность множества всех S-блоков равна $N!$, где $N \in 16, 256$, то для проведения исследования количественных характеристик определенных ранее требований использована бесповторная выборка объемом 10^6 элементов.

Далее собрана статистика по каждой характеристике и предложена вероятность p , заданная как отношение количества элементов, удовлетворяющих критерию, и общего объема выборки.

С целью определения границ задана вероятность $p_{16}=0.01$ для $N=16$ и $p_{256}=0.1$ при $N=256$. Тогда при объеме выборки 10^6 элементов количество элементов, удовлетворяющих какому-либо критерию, будет равно 10000 и 100000 для S-блоков степени 16 и 256 соответственно.

Работа с полученными данными осуществлена следующим образом:

- произведен выбор самого меньшего или большего значения из выборки (в зависимости от конкретного требования);

- осуществлен подбор второго соответствующего граничного значения таким образом, чтобы мощность получаемых подмножеств была близка к 10000 или 100000 соответственно (в зависимости от степени подстановки) или незначительно превышала ее.

Гистограммы выборочного распределения значений критериев для S-блоков степени 16 представлены на рисунках 13-21.

Гистограммы выборочного распределения значений критериев для S-блоков степени 256 представлены на рисунках 22-30.

Для удобства отображения результатов на гистограммах черным цветом выделены те подмножества, которые удовлетворяют критериям, определенным в п.п.2.1-2.6.

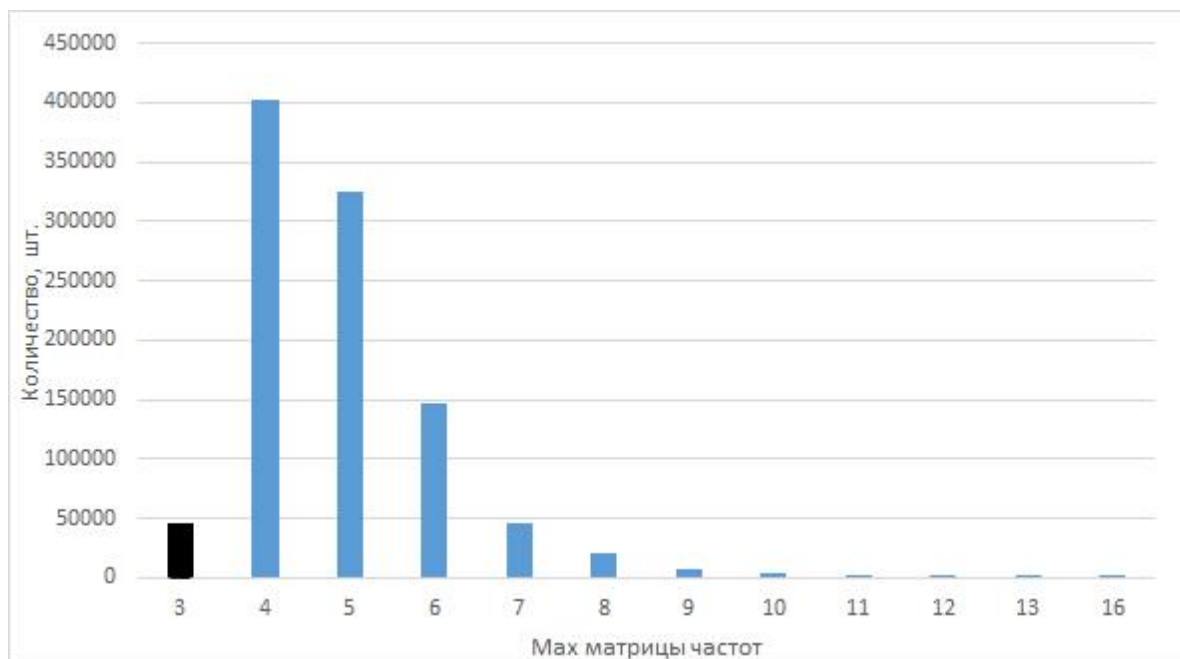


Рисунок 13 – Гистограмма выборочного распределения значений максимумов матрицы частот S-блоков степени 16

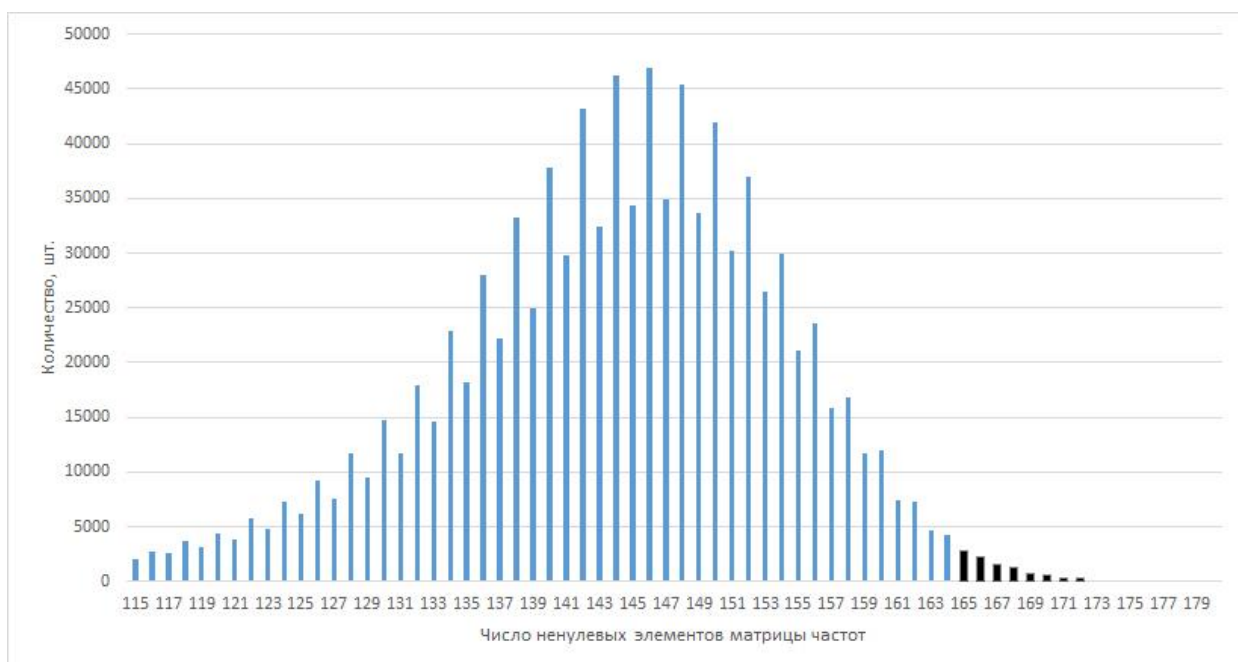


Рисунок 14 – Гистограмма выборочного распределения значений количества ненулевых элементов матрицы частот S-блоков степени 16

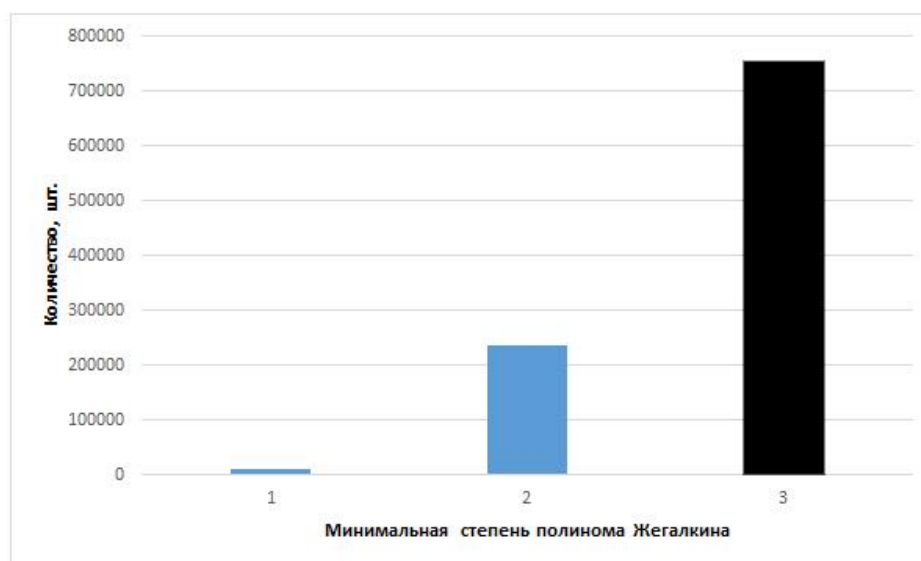


Рисунок 15 – Гистограмма выборочного распределения значений минимальной степени полинома Жегалкина координатных функций S-блоков степени 16

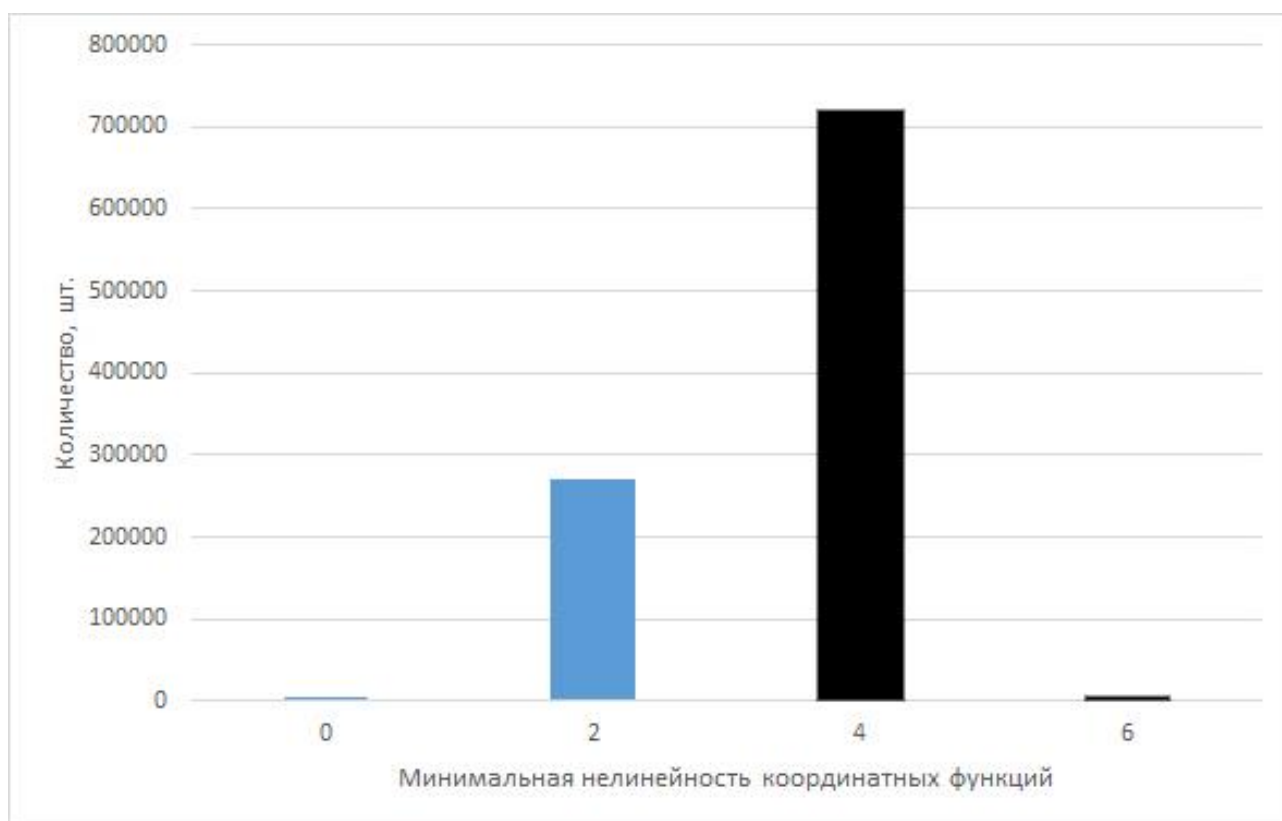


Рисунок 16 – Гистограмма выборочного распределения значений нелинейности координатных функций S-блоков степени 16

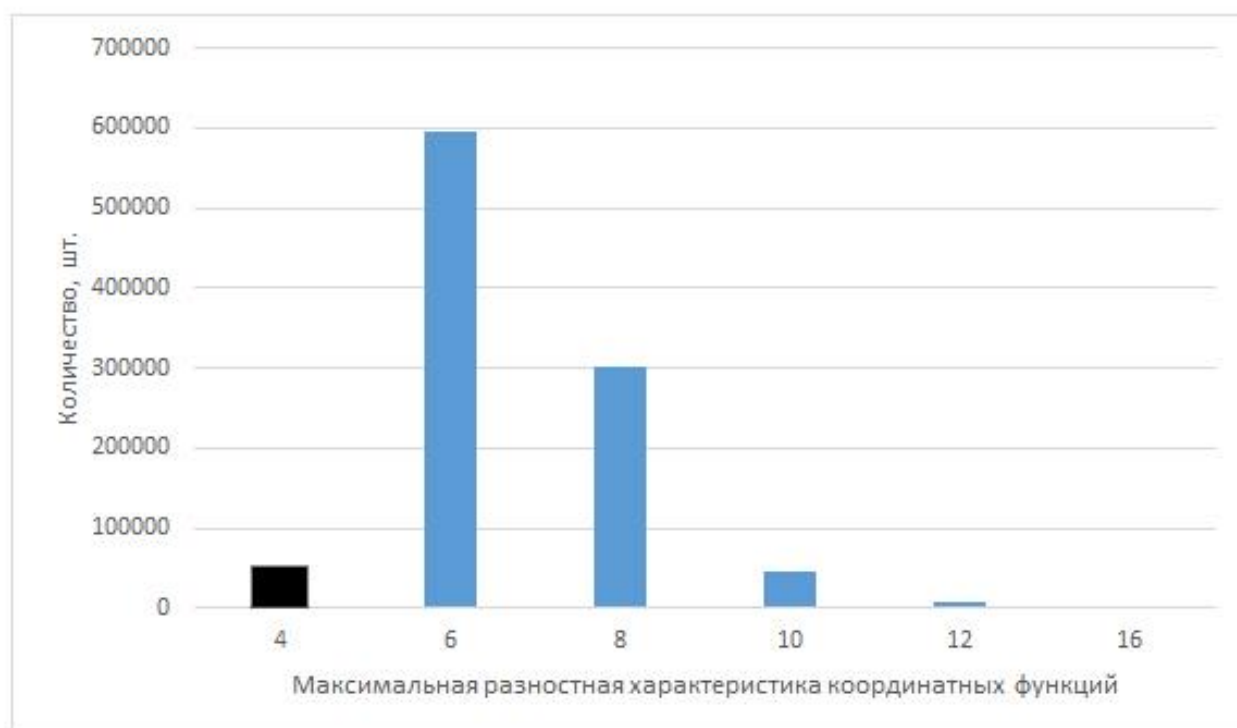


Рисунок 17 – Гистограмма выборочного распределения значений разностной характеристики координатных функций S-блоков степени 16

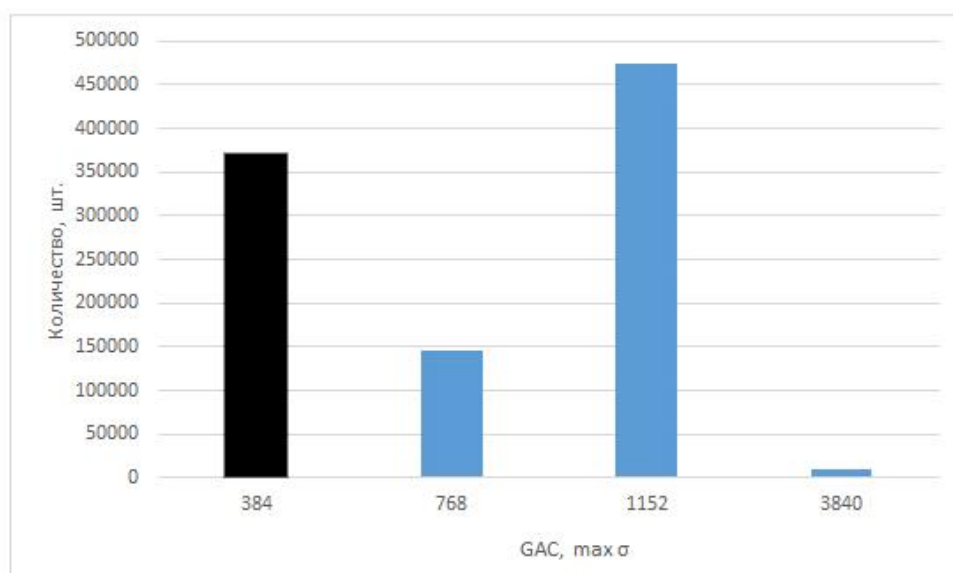


Рисунок 18 – Гистограмма выборочного распределения значений максимумов сумм квадратов автокорреляционных функций по всем направлениям и S-блоков степени 16

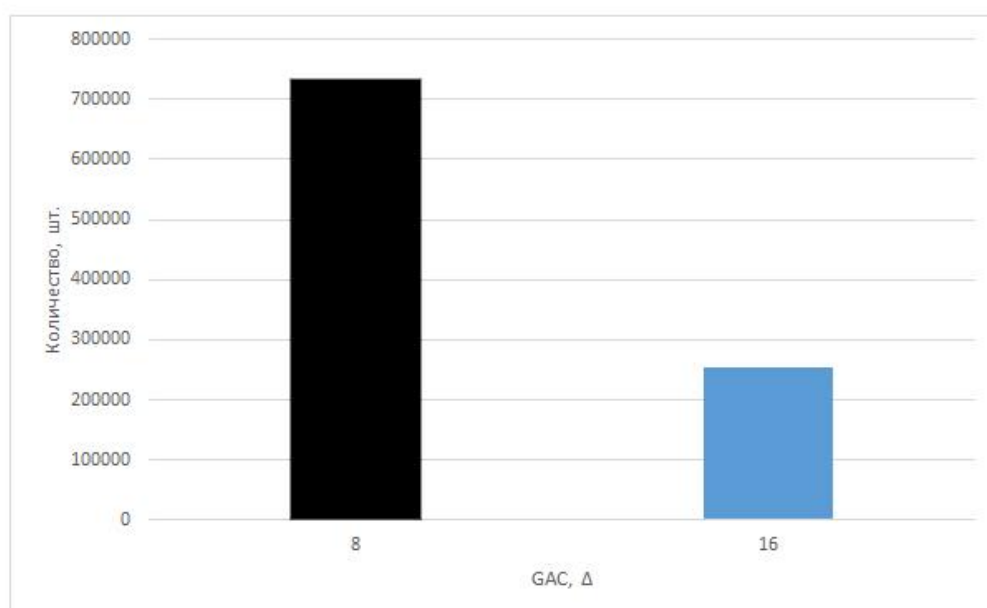


Рисунок 19 – Гистограмма выборочного распределения максимальных значений автокорреляционной функции по всем направлениям и S-блоков степени 16

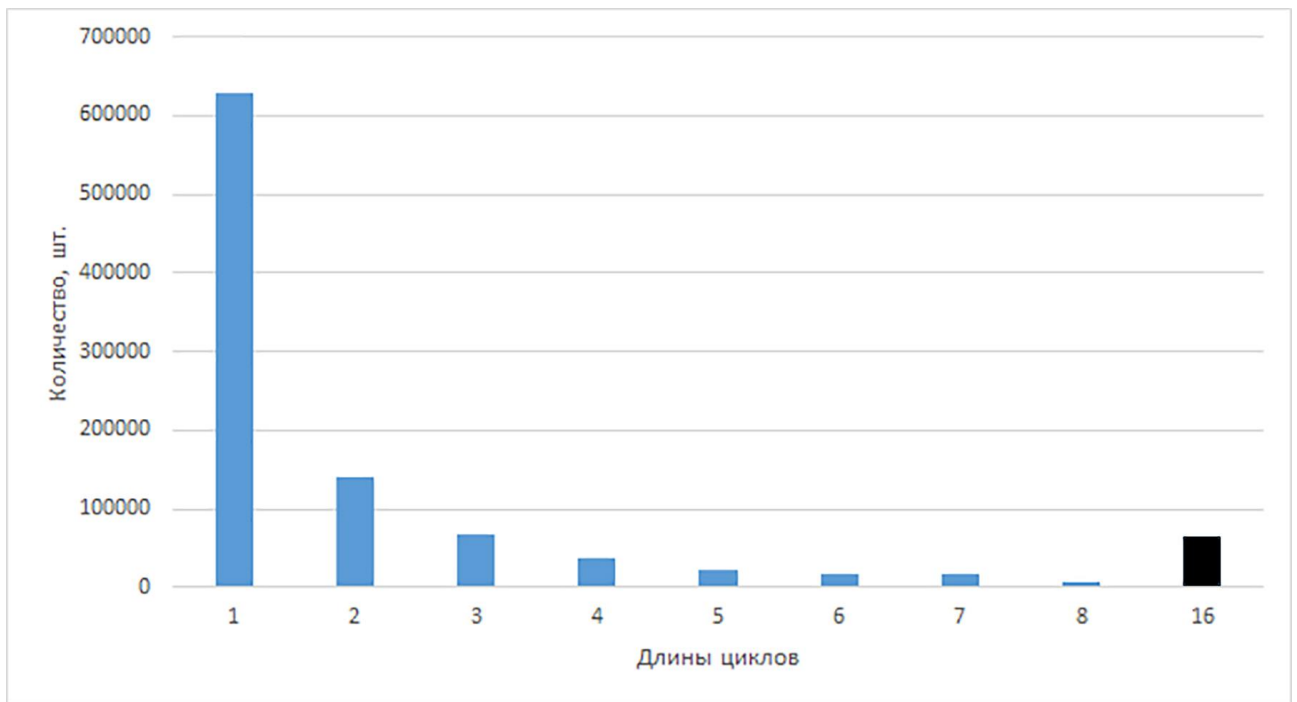


Рисунок 20 – Гистограмма выборочного распределения значений длин циклов S-блоков степени 16

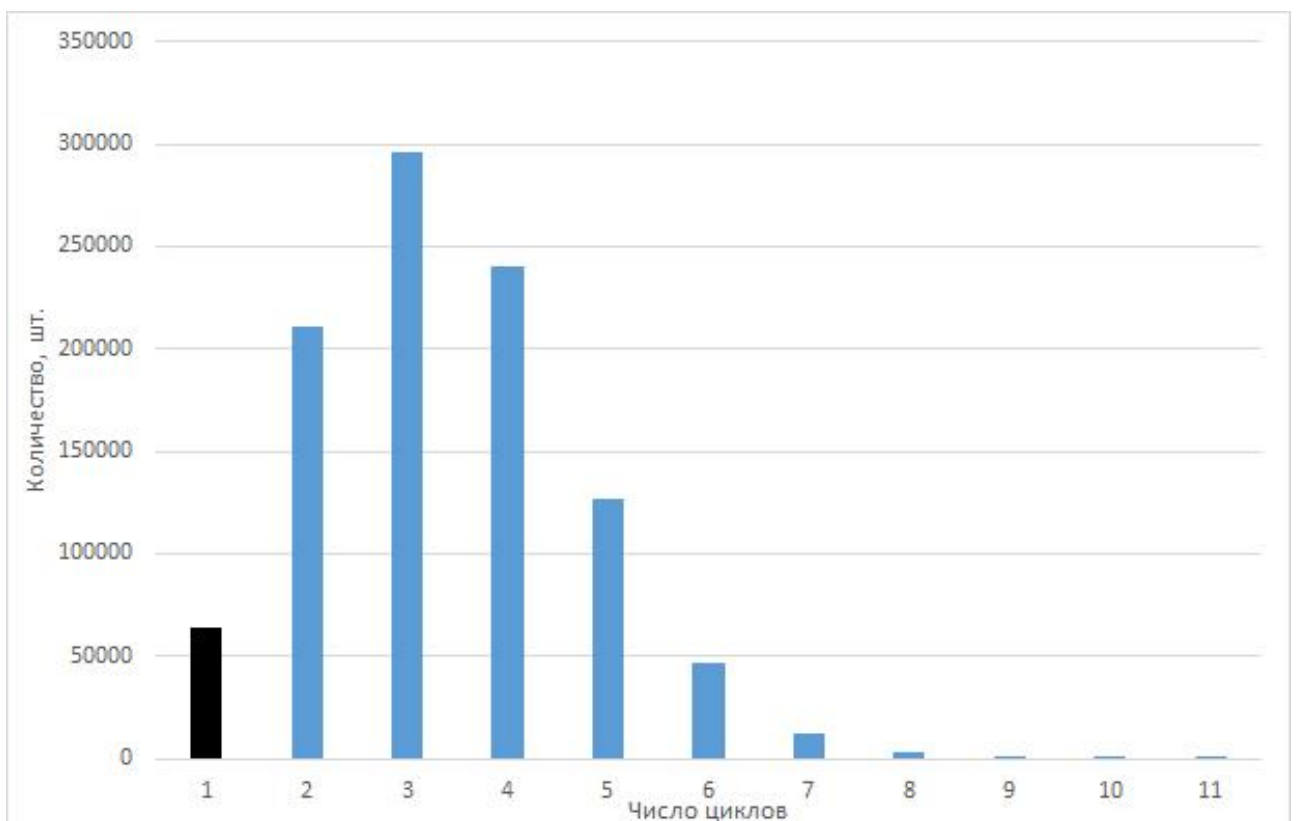


Рисунок 21 – Гистограмма выборочного распределения значений количества циклов S-блоков степени 16

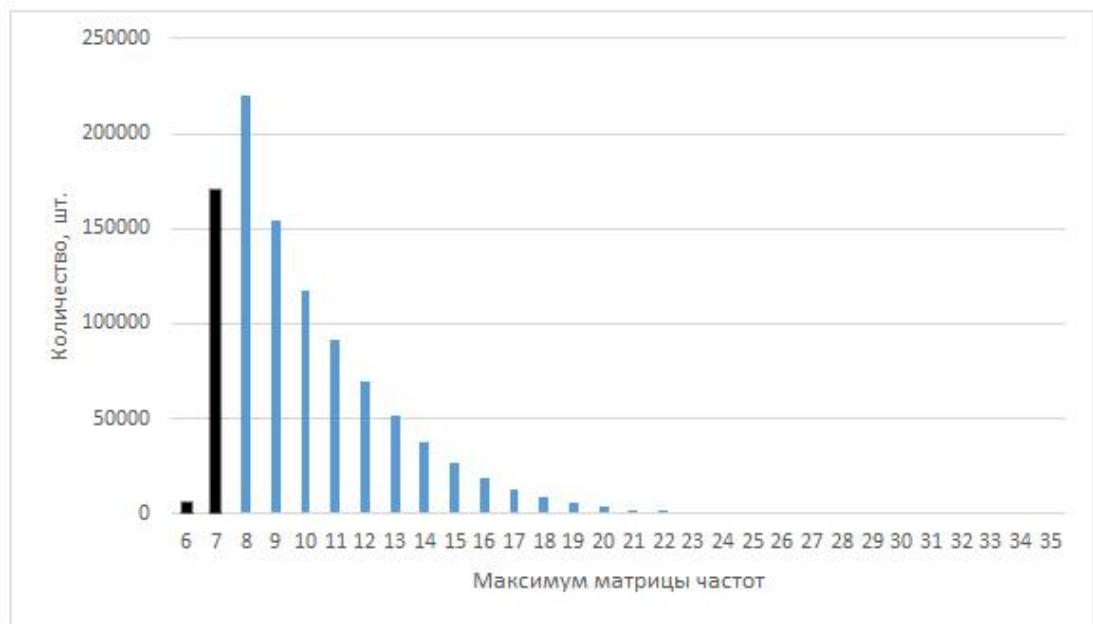


Рисунок 22 – Гистограмма выборочного распределения значений максимумов матрицы частот S-блоков степени 256

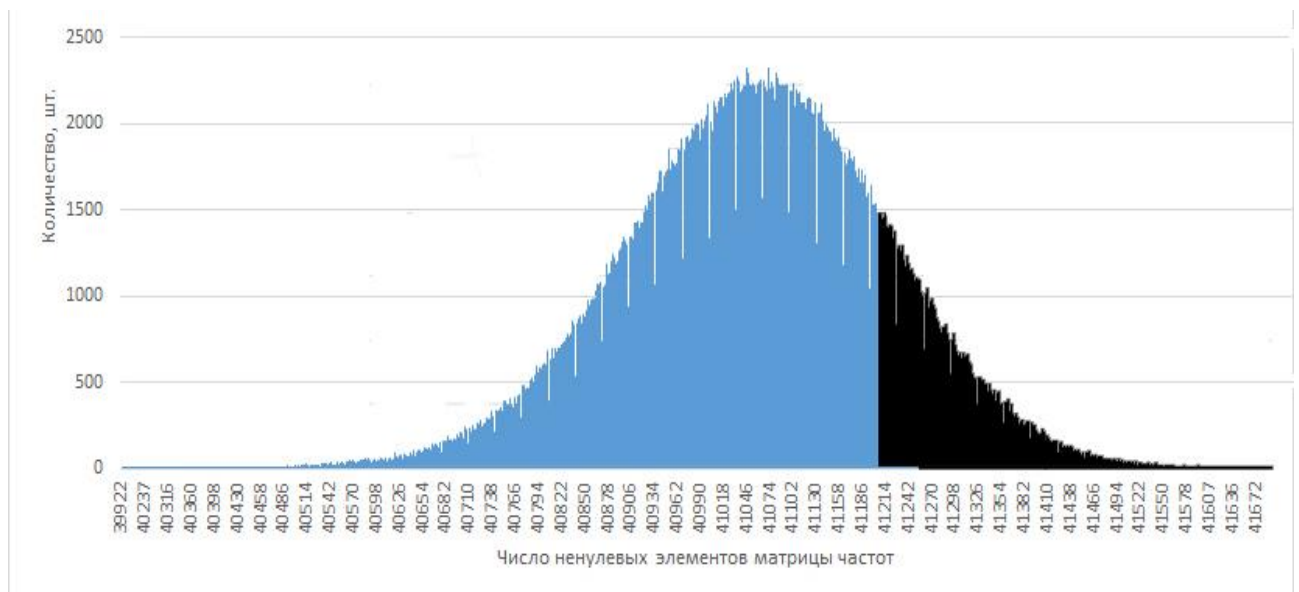


Рисунок 23 – Гистограмма выборочного распределения значений количества ненулевых элементов матрицы частот S-блоков степени 256

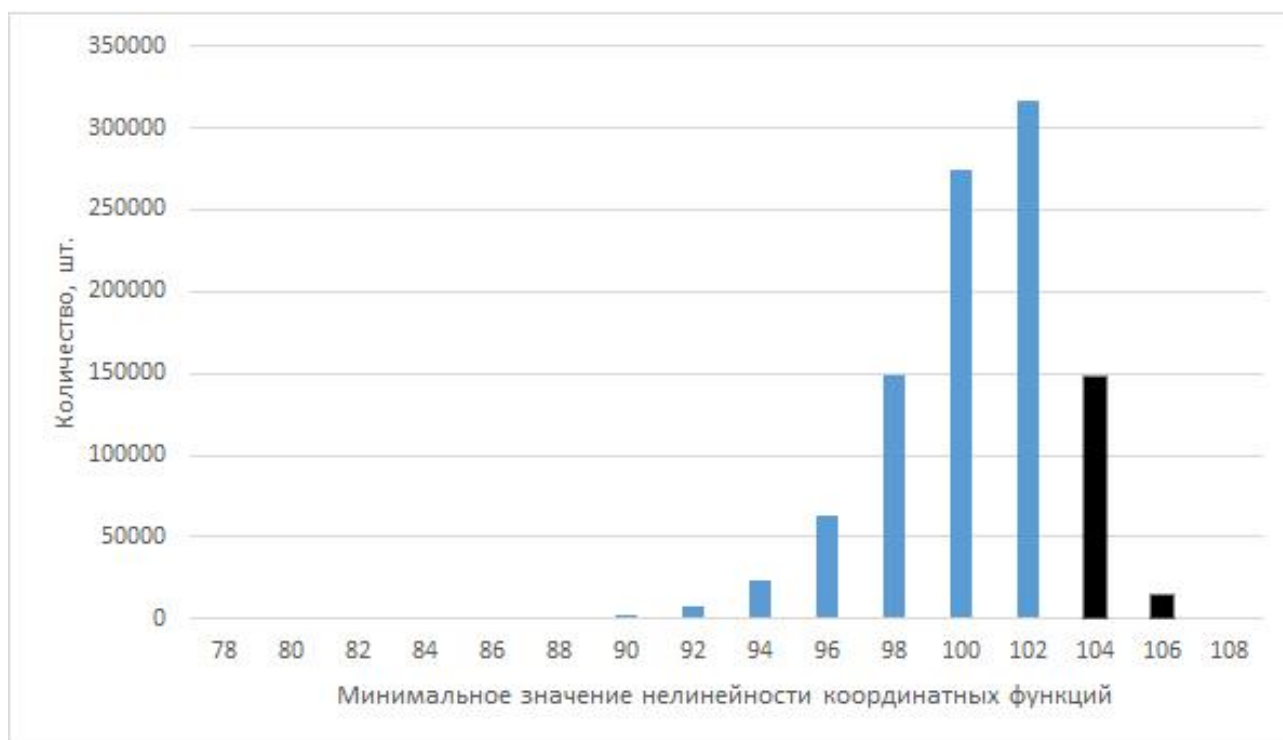


Рисунок 24 – Гистограмма выборочного распределения значений нелинейности координатных функций S-блоков степени 256

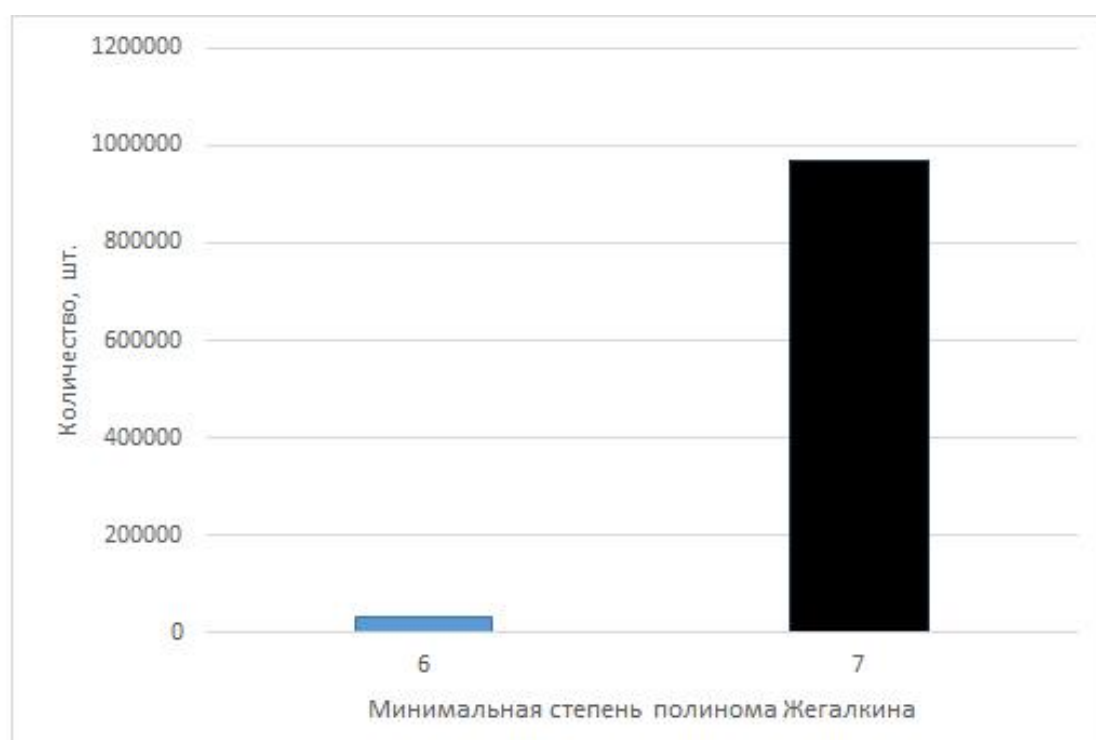


Рисунок 25 – Гистограмма выборочного распределения значений минимальной степени полинома Жегалкина координатных функций S-блоков степени 256

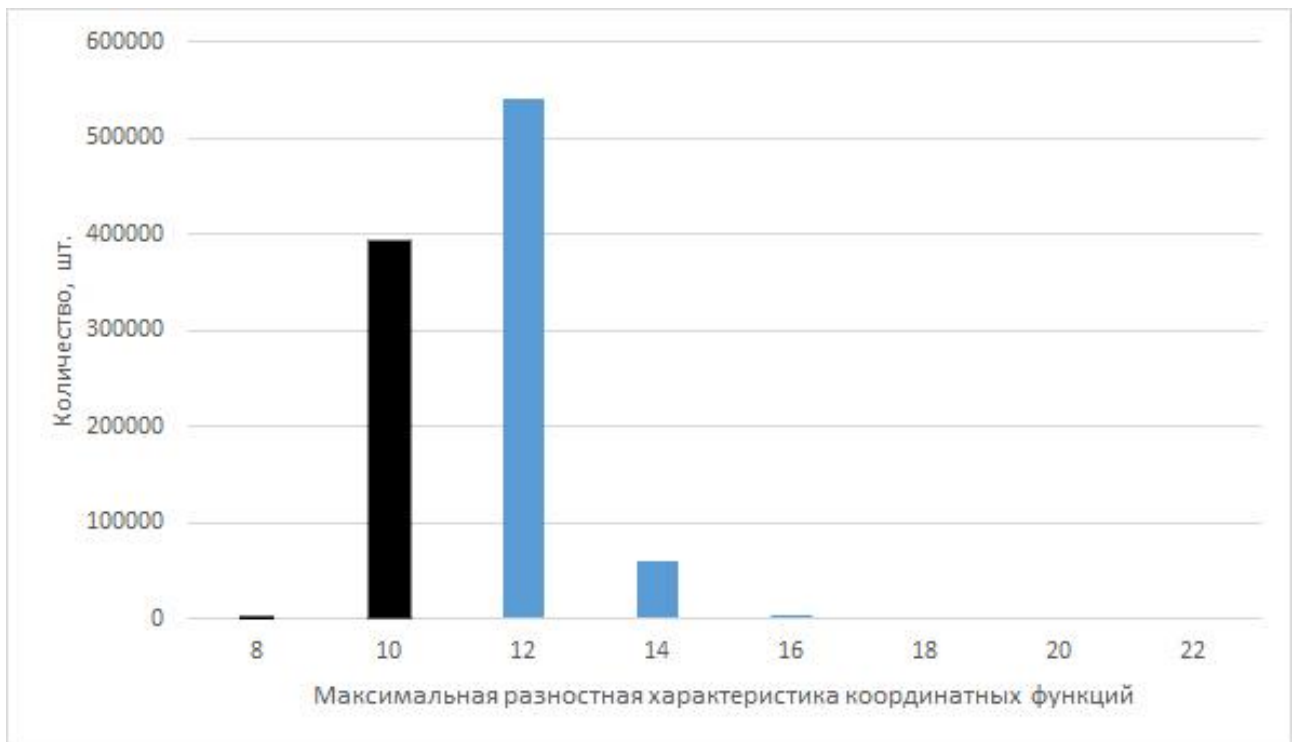


Рисунок 26 – Гистограмма выборочного распределения значений разностной характеристики координатных функций S-блоков степени 256

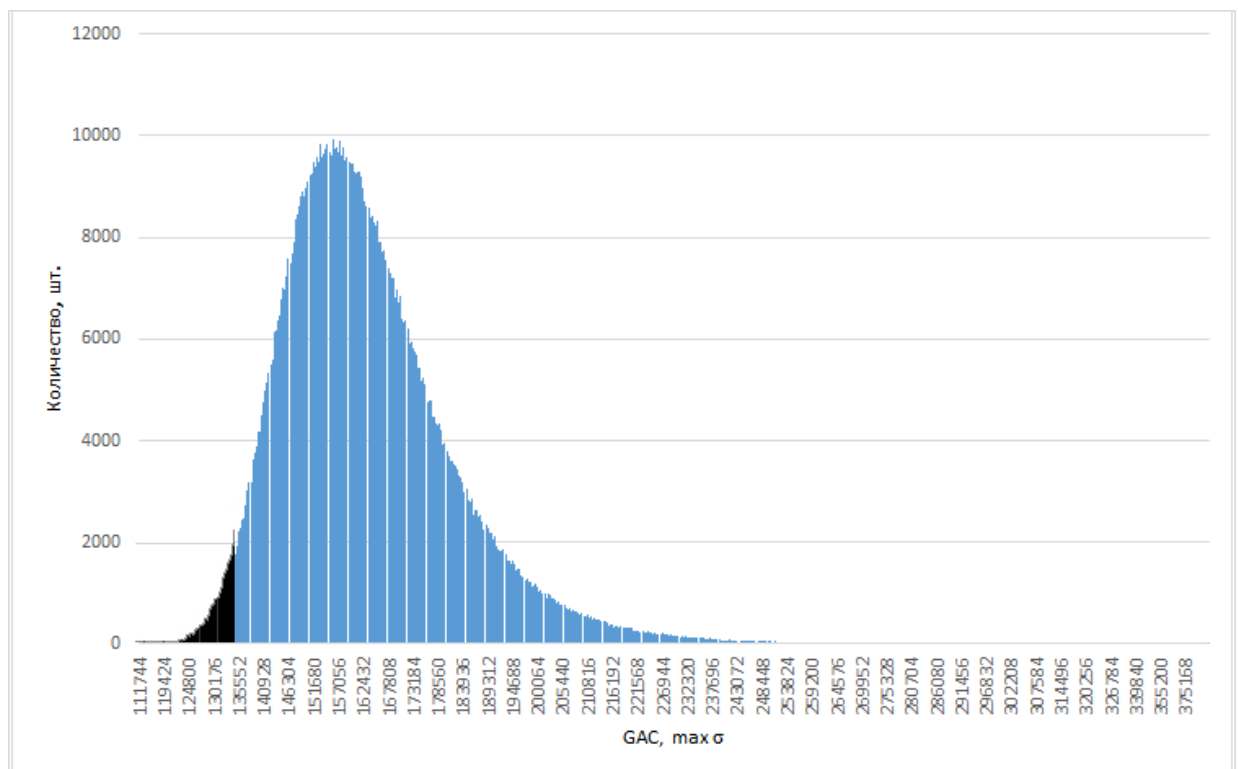


Рисунок 27 – Гистограмма выборочного распределения значений сумм квадратов автокорреляционных функций по всем направлениям и S-блоков степени 256

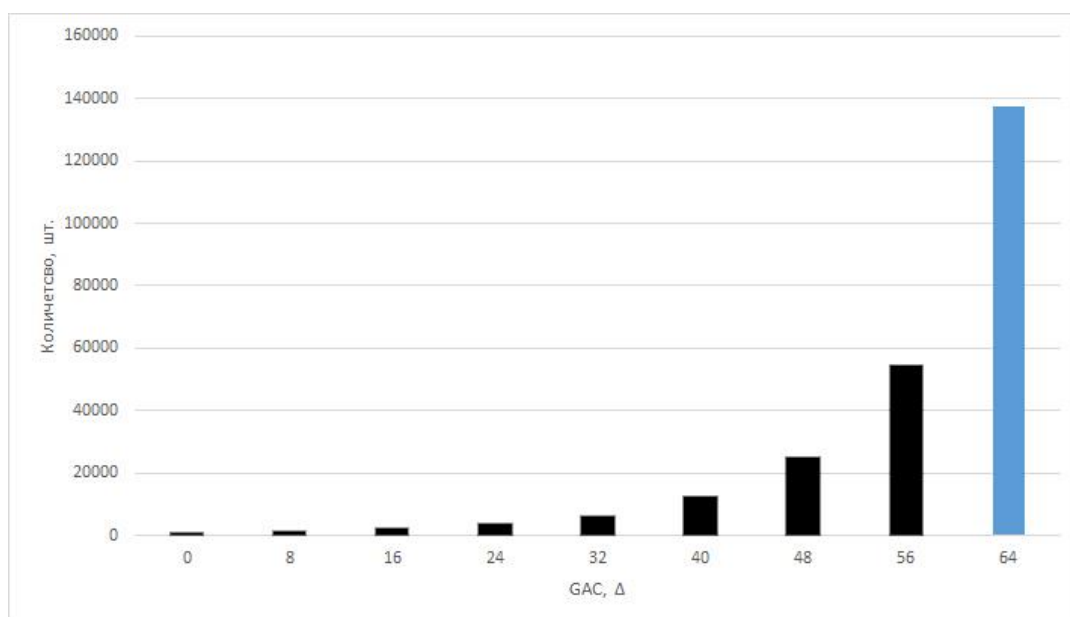


Рисунок 28 – Гистограмма выборочного распределения максимальных значений автокорреляционной функции по всем направлениям и S-блоков степени 256

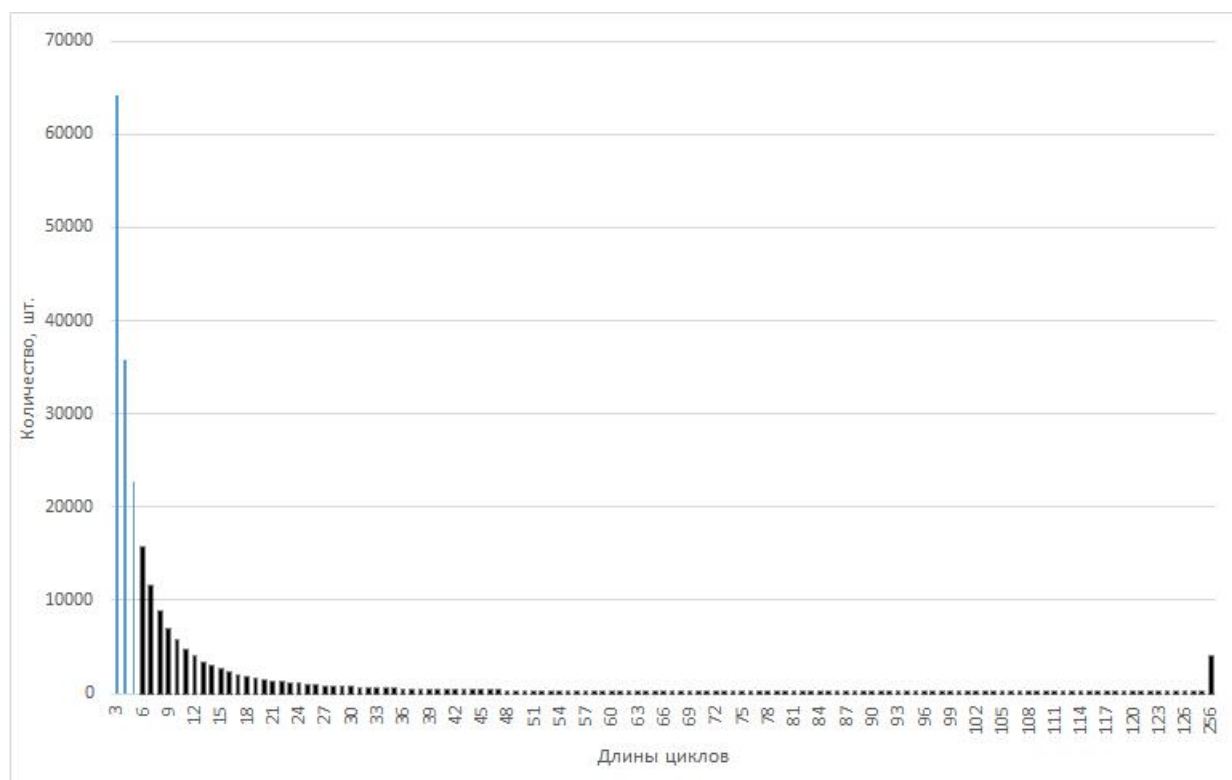


Рисунок 29 – Гистограмма выборочного распределения значений длин циклов S-блоков степени 256

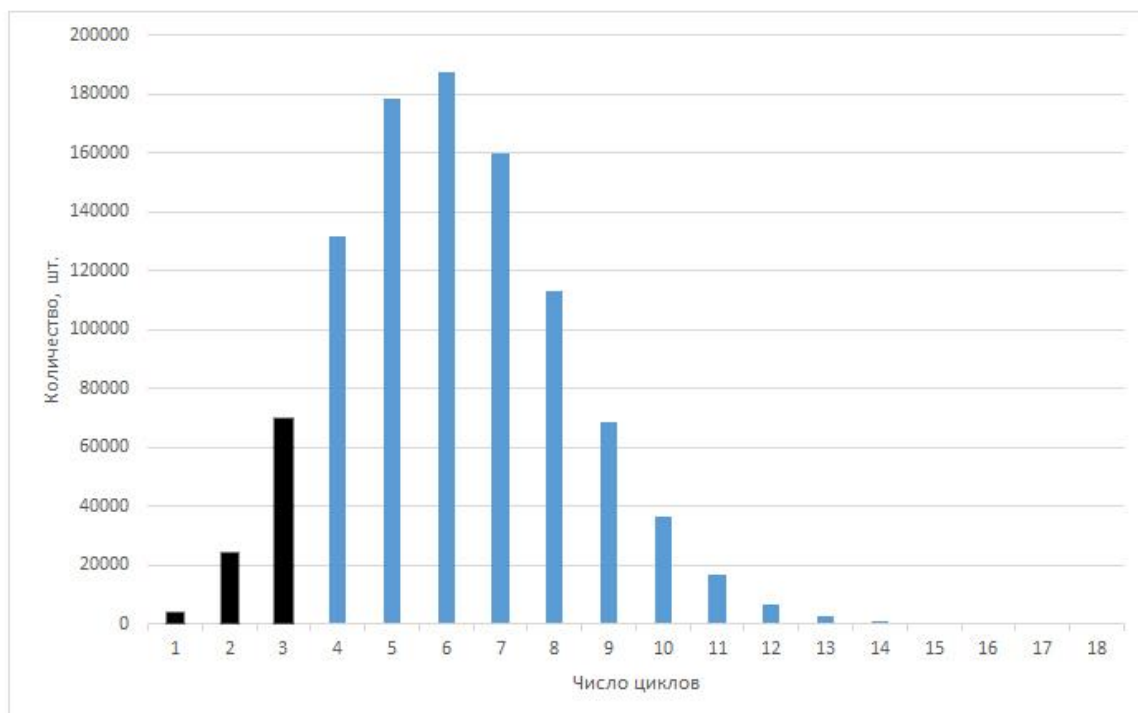


Рисунок 30 – Гистограмма выборочного распределения значений количества циклов S-блоков степени 256

Сводный набор требований, необходимый для обеспечения криптостойкости нелинейного биективного преобразования, с определенными границами значений представлен в таблице 1.

Таблица 1 – Набор требований, необходимый для обеспечения криптостойкости нелинейного биективного преобразования

Характеристика	Требование
Максимальный элемент матрицы встречаемости разностей переходов ненулевых биграмм $P(\pi)$;	$\max_{i,j \neq 0} p_{ij} \leq 7$ при $N=256$ $\max_{i,j \neq 0} p_{ij} \leq 3$ при $N=16$
Число ненулевых элементов матрицы $P(\pi)$	≥ 41200 при $N=256$ ≥ 165 при $N=16$
Максимальная степень нелинейности N_f координатных функций	$104 \leq \max N_{f_i} \leq 120$ при $N=256$ $4 \leq \max N_{f_i} \leq 6$ при $N=16$
Минимальная алгебраическая степень координатных функций	$\min(\deg f_i) \geq 7$ при $N=256$ $\min(\deg f_i) \geq 3$ при $N=16$

Продолжение таблицы 1

Характеристика	Требование
Максимальная разностная характеристика координатных функций	$2 \leq \max \delta_{f_i} \leq 10$ при $N=256$, $2 \leq \max \delta_{f_i} \leq 4$ при $N=16$,
Минимальная длина циклов, представляющих S-блок	≥ 16 при $N=256$ $= 16$ при $N=16$
Максимальное количество циклов	≤ 3 при $N=256$, $= 1$ при $N=16$
отсутствие неподвижных точек S-блока	-
Глобальный лавинный критерий	
Максимальная сумма квадратов значений автокорреляционной функции по всем направлениям u	$\max \sigma_{f_i} \leq 132096$ при $N=256$ $\max \sigma_{f_i} \leq 384$ при $N=16$
Максимальное значение автокорреляционной функции по всем направлениям u , кроме $u \neq 0$:	$\max \Delta_{f_i} \leq 56$ при $N=256$ $\max \Delta_{f_i} \leq 8$ при $N=16$

3 Разработка алгоритма генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности

Принцип «доказуемой» псевдослучайности связан с генерацией значений каких-либо параметров с использованием базового криптографического механизма и известных начальных значений.

Согласно техническому заданию на дипломный проект, данным механизмом использован алгоритм блочного шифрования «Кузнечик», представленный в ГОСТ Р 34.12-2015, в режиме гаммирования.

При реализации алгоритма генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности для работы с алгоритмом блочного шифрования «Кузнечик» в режиме гаммирования необходимы начальные значения мастер-ключа для построения итерационных ключей и вектора инициализации для создания синхропосылки.

Ввод начальных значений мастер-ключа и вектора инициализации для удобства работы с программным обеспечением будет производиться при помощи латинских символов и цифр, которые затем в ПО будут представлены в виде 16-битовых чисел кодировки ASCII.

Алгоритм блочного шифрования «Кузнечик» имеет следующие основные параметры: длина блока – 128 бит, длина ключа – 256 бит.

Данный алгоритм построен по принципу подстановочно-перестановочной сети (SP-сети).

В алгоритме зашифрования и расшифрования блочного шифра с размером блока 128 бит (шифр «Кузнечик») и размером ключа 256 бит используются преобразования, показанные в таблице 2.

Таблица 2 – Преобразования, используемые в шифре «Кузнечик»

Преобразование	Описание
$X[k]: V_{128} \rightarrow V_{128}$	$X[k](a) = k \bmod 2, \text{ где } k, a \in V_{128}$
$S: V_{128} \rightarrow V_{128}$	$S(a) = S(a_{15} \dots a_0) = \pi(a_{15}) \dots \pi(a_0)$, где $a = a_{15} \dots a_0 \in V_{128}, a_i \in V_{128}, i = 0, 1, \dots, 15$
$S^{-1}: V_{128} \rightarrow V_{128}$	преобразование, обратное к преобразованию S , которое может быть вычислено следующим образом: $S(a) = S(a_{15} \dots a_0) = \pi(a_{15}) \dots \pi(a_0)$ где π^{-1} , подстановка, обратная подстановке π , $a = a_{15} \dots a_0 \in V_{128}, a_i \in V_{128}, i = 0, 1, \dots, 15$
$R: V_{128} \rightarrow V_{128}$	$R(a) = R(a_{15} \parallel \dots \parallel a_0) = l(a_{14}, a_{13}, \dots, a_0, a_{15}) \parallel a_{15} \parallel a_{14} \parallel \dots \parallel a_1$, где $a = a_{15} \dots a_0 \in V_{128}, a_i \in V_{128}, i = 0, 1, \dots, 15$
$L: V_{128} \rightarrow V_{128}$	$L(a) = R^{16}(a), \text{ где } a \in V_{128}$
$R^{-1}: V_{128} \rightarrow V_{128}$	преобразование, обратное к преобразованию R , которое может быть вычислено следующим образом: $R^{-1}(a) = R^{-1}(a_{15} \dots a_0) = a_{14} \parallel a_{13} \parallel \dots \parallel a_0 \parallel l(a_{14}, a_{13}, \dots, a_0, a_{15})$, где $a = a_{15} \dots a_0 \in V_{128}, a_i \in V_{128}, i = 0, 1, \dots, 15$
$L^{-1}: V_{128} \rightarrow V_{128}$	$L^{-1}(a) = (R^{-1})^{16}(a), \text{ где } a \in V_{128}$
$F[k]: V_{128} \times V_{128} \rightarrow V_{128} \times V_{128}$	$F[k](a_1, a_0) = (LSX[k](a_1) \bmod 2 a_0, a_1), \text{ где } k, a_0, a_1 \in V_{128}$

Значения параметров определяются при помощи:

– нелинейного биективного преобразования. В его качестве выступает подстановка $\pi = \text{Vec}_8 \pi' \text{Int}_8: V_8 \rightarrow V_8$. Значения подстановки π' показаны на рисунке 5.

– линейного биективного преобразования. Оно задается отображением

$l: V_8^{16} \rightarrow V_8$, которое определяется следующим образом:

$$l(a_{15}, \dots, a_0) = \nabla (148 \cdot \Delta(a_{15}) + 32 \cdot \Delta(a_{14}) + 133 \cdot \Delta(a_{13}) + 16 \cdot \Delta(a_{12}) + 194 \cdot \Delta(a_{11}) + (12)$$

$$+ 192 \cdot \Delta(a_{10}) + 1 \cdot \Delta(a_9) + 251 \cdot \Delta(a_8) + 1 \cdot \Delta(a_7) + 192 \cdot \Delta(a_6) + 194 \cdot \Delta(a_5) + 16 \cdot \Delta(a_4) + \\ + 133 \cdot \Delta(a_3) + 32 \cdot \Delta(a_2) + 148 \cdot \Delta(a_1) + 1 \cdot \Delta(a_0)$$

Блочный шифр размером блока 128 бит и размером ключа 256 бит состоит из следующих частей:

– алгоритм развертывания ключа. Использует итерационные константы $C_i \in V_{128}$, где $i = 1, 2, 3, \dots, 32$; V_{128} – множество всех двоичных строк длины 128 бит. Итерационные константы определены следующим образом: $C_i = L(\text{Vec}_{128}(i)), i = 1, 2, \dots, 32$. Итерационные ключи $K_i \in V_{128}, i = 1, 2, \dots, 10$ вырабатываются на основе ключа $k = k_{255} \dots k_0 \in V_{256}, k_i \in V_1, i = 1, 2, \dots, 255$ и определяются равенствами: $k1 = k_{255} \parallel \dots \parallel k_{128}; k2 = k_{127} \parallel \dots \parallel k_0$;
 $(K_{2i+1}, K_{2i+2}) = F[C_{8(i-1)+8}] \dots F[C_{8(i-1)+1}](K_{2i-1}, K_{2i}), i = 1, 2, 3, 4$.

– алгоритм зашифрования;

Алгоритм зашифрования в зависимости от значений итерационных ключей $K_i \in V_{128}, i = 1, 2, \dots, 10$ реализует подстановку $E_{K_1, \dots, K_{10}}$, заданную на множестве V_{128} , в соответствии с равенством:
 $D_{K_1, \dots, K_{10}}(a) = X[K_1]L^{-1}S^{-1}X[K_2] \dots L^{-1}S^{-1}X[K_9]L^{-1}S^{-1}X[K_{10}](a), \text{ где } a \in V_{128}$.

– алгоритм расшифрования;

Алгоритм расшифрования в зависимости от значений итерационных ключей $K_i \in V_{128}, i = 1, 2, \dots, 10$ реализует подстановку $D_{K_1, \dots, K_{10}}$, заданную на множестве V_{128} , в соответствии с равенством:
 $E_{K_1, \dots, K_{10}} = X[K_{10}]LSX[K_9] \dots LSX[K_2]LSX[K_1](a), \text{ где } a \in V_{128}$.

В соответствии с техническим заданием алгоритм блочного шифрования «Кузнечик» был использован в режиме гаммирования.

Для работы алгоритма «Кузнечик» в данном режиме помимо мастер-ключа необходим вектор синхронизации IV.

Схема работы алгоритма блочного шифрования в режиме гаммирования представлен на рисунке 31.

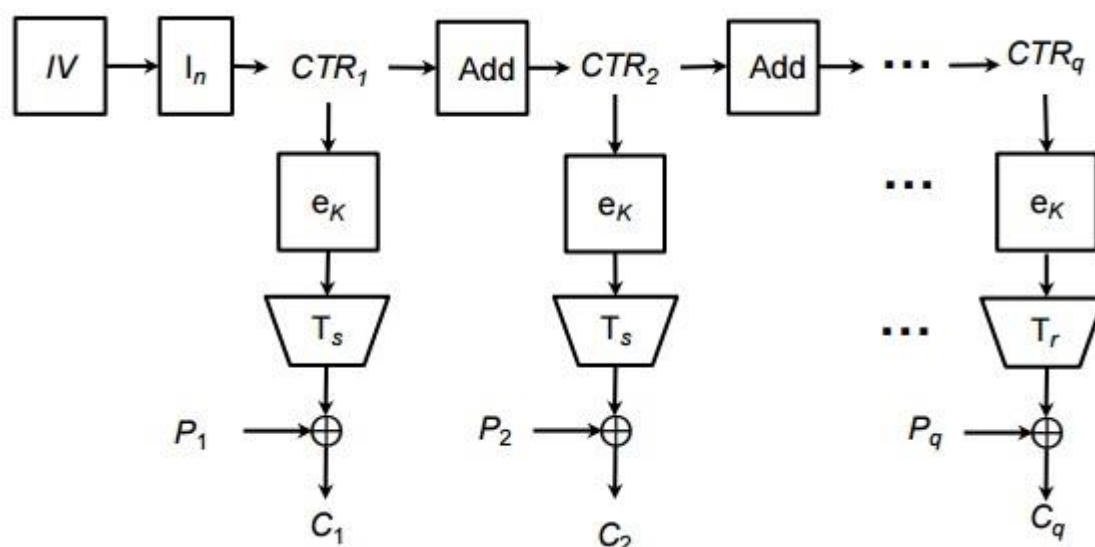


Рисунок 31 – Режим гаммирования

Вектор инициализации $IV \in V_{64}$ при помощи операции I_n дополняется нулевыми битами справа до длины 128 бит.

Далее полученное значение счетчика зашифровывается алгоритмом блочного шифрования «Кузнечик» в режиме простой замены и, гамма накладывается на открытый текст P_i . При необходимости может использоваться операция T_s , которая предназначена для усечения гаммы до длины s . Т.к. $s=128$, то такая операция выполняться не будет.

В настоящем дипломном проекте режим гаммирования работает «вхолостую», т.е. наложения выработанной гаммы на открытый текст не происходит. Вместо этого из полученной на i -ой итерации гаммы выбирается младший байт, на который накладывается единичная битовая маска, длина которой зависит от степени искомого S-блока: 16 или 256. Выработка элементов S-блока происходит циклически, а длина такого цикла также зависит от степени искомого S-блока.

После выполнения описанной i -ой итерации $i \in \overline{1, N}$, где $N \in 16, 256$, полученное значение присваивается i -ому элементу генерируемого S-блока.

В случае если полученное значение элемента S-блока было получено ранее, то происходит сложение данного значения с элементом 7. Операция сложения происходит в кольце вычетов Z_N , где $N \in 16, 256$.

Таким образом, разработан алгоритм генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности, показанный на рисунке 32.

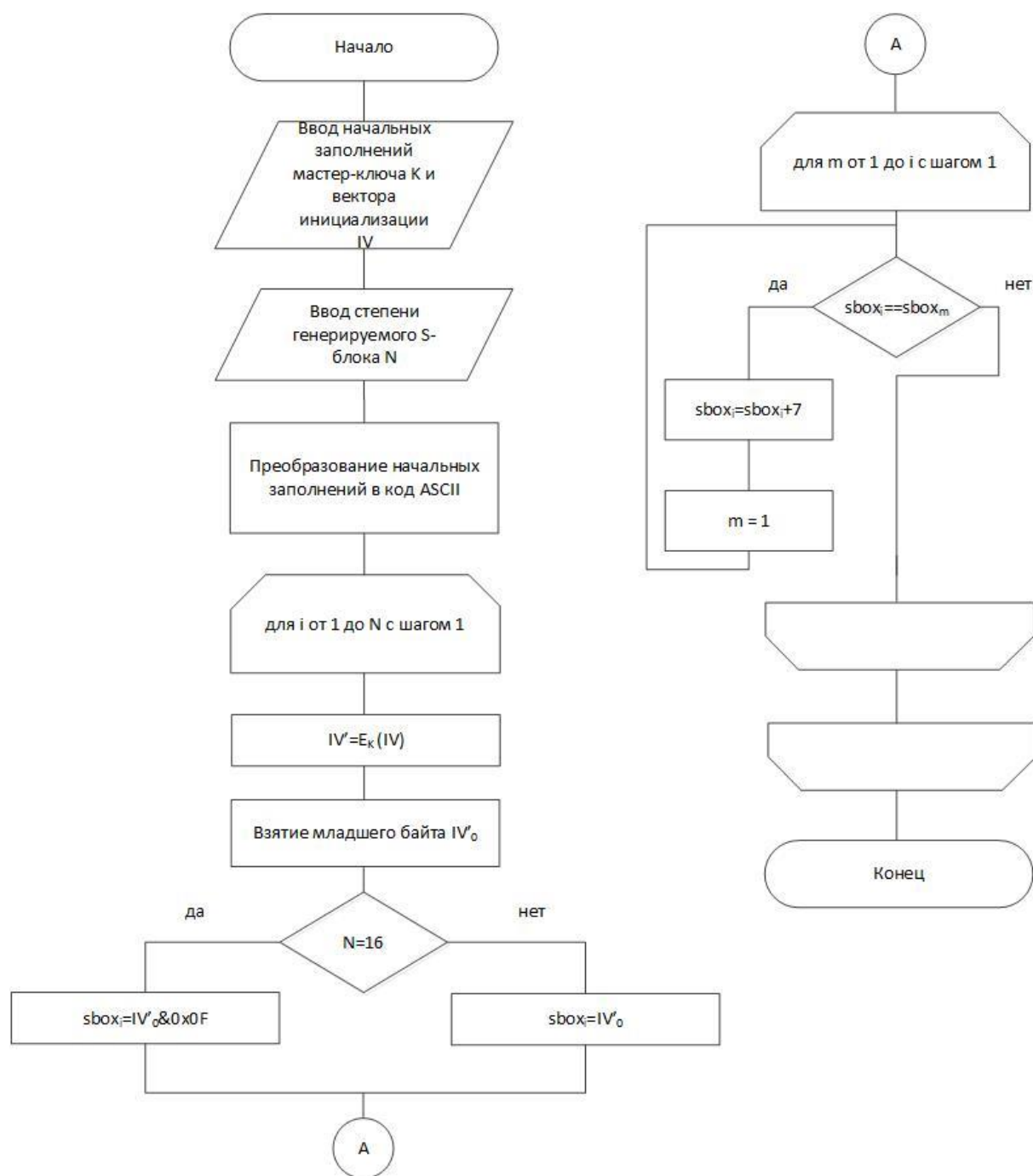


Рисунок 32 – Алгоритм генерации узлов замены

4 Разработка методики и рекомендации по работе с программным обеспечением

После определения списка требований, необходимых для обеспечения криптостойкости нелинейного биективного преобразования и построения алгоритма генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности разработано программное обеспечение «Нелинейное биективное преобразование».

Данное приложение создано при помощи кроссплатформенного инструментария разработки ПО Qt. Разработка программного обеспечения осуществлена на языке высокого уровня C++.

В рамках разработки программного обеспечения «Нелинейное биективное преобразование» также создан графический интерфейс, обеспечивающий более удобное взаимодействие между ЭВМ и пользователем.

Данное приложение работает на операционных системах семейства Windows, начиная с версии 7 и выше.

Для начала работы с данным приложением необходимо в директории программы запустить исполняемый файл s-box.exe.

В панели меню «Режимы работы» можно выбрать один из режимов работы данного программного обеспечения:

- поиск подмножества нелинейных биективных преобразований, соответствующих заданным требованиям;
- проверка выбранного нелинейного биективного преобразования на соответствие заданным требованиям.

Также выбор одного из этих режимов можно осуществить по «горячим» клавишам:

- режим поиска подмножества нелинейных биективных преобразований, соответствующих заданным требованиям, может быть вызван при помощи сочетания клавиш «Ctrl+1»;

- режим проверки выбранного нелинейного биективного преобразования на соответствие заданным требованиям может быть вызван при помощи сочетания клавиш «Ctrl+2».

В первом режиме для осуществления поиска подмножества S-блоков, соответствующих заданным требованиям, необходимо произвести следующие действия:

- определить путь сохранения найденных S-блоков при помощи кнопки «Выбор пути». Файлом сохранения может быть существующий или созданный файл формата *.txt. Следует обратить внимание на то, что при выборе существующего файла в качестве файла сохранения все предыдущее содержание текстового файла будет удалено;

- определить степень искомых S-блоков. Доступно два варианта выбора степени: 16 и 256;

- в соответствующие текстовые поля ввести начальные заполнения мастер-ключа и вектора инициализации. В качестве символов заполнения могут выступать любые символы, кроме символов кириллицы;

- для запуска поиска S-блоков необходимо нажать кнопку «Начать поиск».

В случае отсутствия каких-либо заполненных параметров при нажатии кнопки «Начать поиск» выводится предупреждение о незаполненных данных, и запуск поиска не осуществляется.

Внешний вид приложения в данном режиме представлен на рисунке 33.

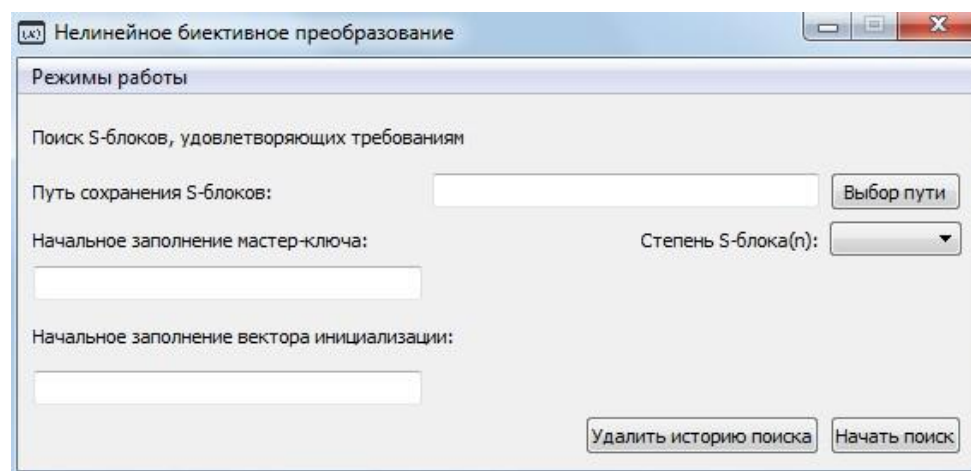


Рисунок 33 – Режим поиска подмножества нелинейных биективных преобразований, соответствующих заданным требованиям

База данных программного обеспечения необходима для хранения информации о проводимых ранее поисках S-блоков. Хранение базы данных осуществляется в формате JSON. При этом сохраняются следующие данные:

- IV_CTR_current – значение синхропосылки в момент приостановки поиска;
- IV_CTR_start – начальное заполнение вектора инициализации, введенное пользователем;
- count_sbox – число найденных S-блоков, которые удовлетворяют требованиям;
- count_search – общее число проверенных S-блоков;
- master_key_current – значение мастер-ключа в момент приостановки поиска;
- master_key_start – начальное заполнение мастер-ключа, введенное пользователем;
- sizeofbox- степень S-блока, при которой проводился поиск.

При отсутствии по какой-либо причине файла БД в формате json в директории «config» будет выведено предупреждающее сообщение о повреждении log-файла.

При помощи кнопки «Удалить историю поиска» можно очистить все записи о произведенных ранее поисках S-блоков. Удаление истории поиска возможно только при условии, что процесс нахождения S-блоков не запущен.

После нажатия кнопки «Начать поиск» на экране программы, производится проверка введенных пользователем начальных заполнений на совпадение хранимых значений в базе данных программы.

В случае определения того, что поиск при заданных начальных значениях уже проводился ранее, появится диалоговое окно, пример которого представлен на рисунке 34.

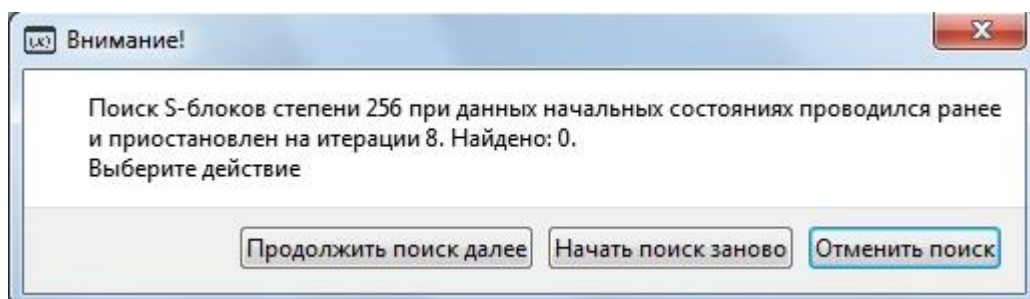


Рисунок 34 – Пример предупреждения о проведении поиска с одними и теми же заполнениями мастер-ключа и вектора инициализации

При этом предлагаются следующие варианты:

- «Продолжить поиск далее». В этом случае поиск будет продолжен с того места, на котором он был прерван ранее;
- «Начать поиск заново». В этом случае поиск будет начат с самого начала;
- «Отменить поиск». В этом случае поиск будет отменен, и пользователю будет предоставлена возможность изменения параметров поиска.

В случае успешного прохождения проверки введенных пользователем начальных заполнений запустится процесс поиска.

Процесс поиска S-блоков представлен на рисунке 35.

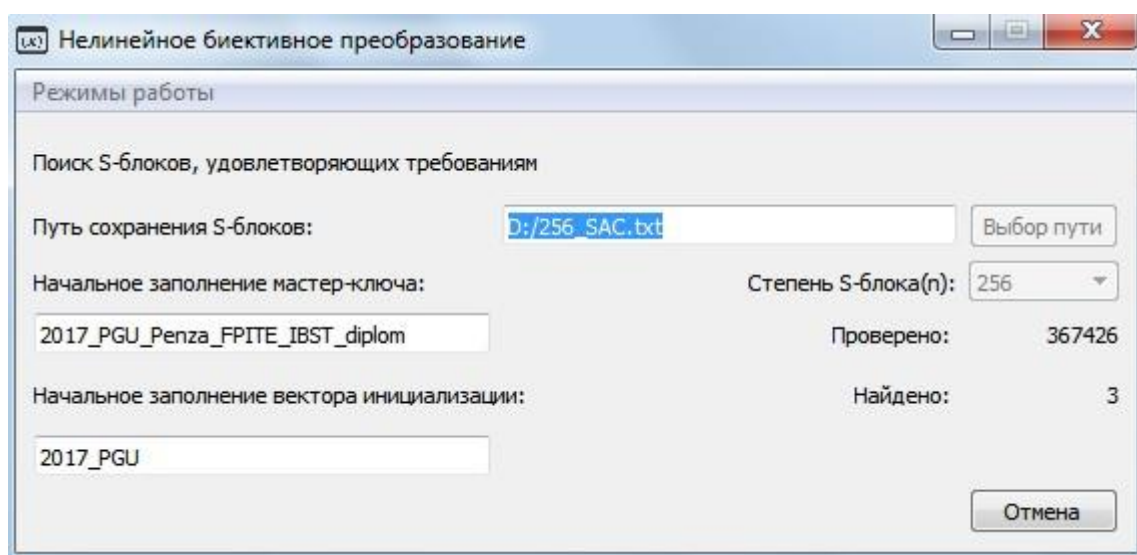


Рисунок 35 – Процесс поиска S-блоков

В процессе поиска S-блоков появляются пункты «Проверено» и «Найдено» с целью предоставления пользователю информации о текущей ситуации.

В файл, указанный в графе «Путь сохранения S-блоков», помимо самого S-блока записывается информация, касаемая его характеристик, определенных в разделе 2. Эта информация состоит из:

- описания характеристики с выводом о ее соответствии или несоответствии (например, «*Минимальная степень полинома Жегалкина выше или равно порогу: да*»);
- значения характеристики (например, «*Значение: 7*»).

Сохранение результатов поиска возможно только по нажатию кнопки «Отмена». В случае попытки закрытия приложения во время запущенного режима поиска S-блоков появляется предупреждающее сообщение, показанное на рисунке 36.

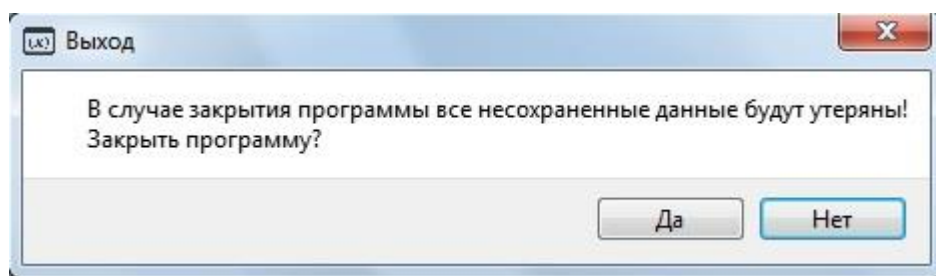


Рисунок 36 – Предупреждающее сообщение о потере несохраненных данных после возможного закрытия приложения

Во втором режиме для осуществления режим проверки выбранного нелинейного биективного преобразования на соответствие заданным требованиям может быть вызван по сочетанию клавиш «Ctrl+2».

Внешний вид приложения в данном режиме представлен на рисунке 37.

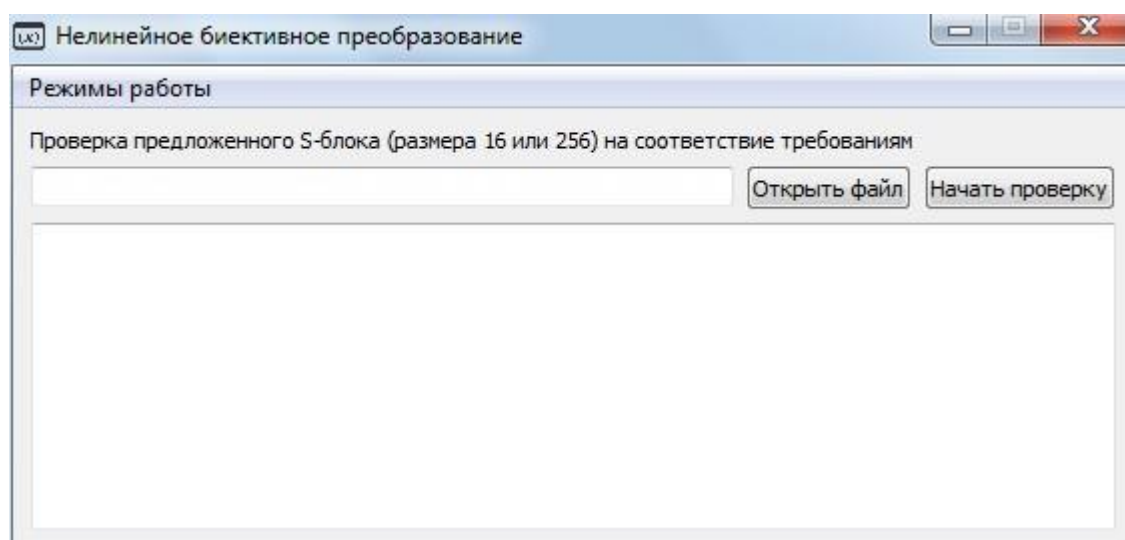


Рисунок 37 – Режим проверки выбранного нелинейного биективного преобразования на соответствие заданным требованиям

Для проведения проверки S-блока на соответствие заданным требованиям должен быть выбран файл формата *.txt, его содержащий. При этом значения вводимого S-блока могут быть либо только в десятичной, либо только в шестнадцатеричной системе счисления (например, 0xab). В качестве разделителей значений S-блока выступают знак пробела, знак табуляции, знак новой строки. В файле должен содержаться только один S-блок.

S-блок, содержащийся в файле, может быть записан любым количеством строк.

При нажатии кнопки «Начать проверку» в начале происходит проверка предложенного файла на соответствие вышеизложенных требований.

В случае, если при извлечении данных из файла возникнет ошибка (количество чисел не равно 16 или 256, присутствуют символы, отличные от цифр), выводится сообщение, представленное на рисунке 38.

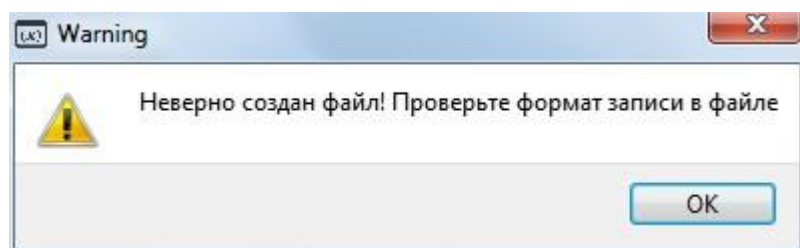


Рисунок 38 – Предупреждение о неверно созданном файле

В случае прохождения проверки на правильность оформления в поле выводится результат проверки S-блока на соответствие заданным требованиям с указанием полученных характеристик.

Данная информация состоит из:

- степени предложенного на проверку S-блока;
- описания характеристики с выводом о ее соответствии или несоответствии (например, «*Минимальная степень полинома Жегалкина выше или равно порогу: да*»);
- значения характеристики (например, «*Значение: 7*»).

Исходный код программного обеспечения представлен в Приложении А.

5 Результаты проведения исследования по поиску S-блоков, удовлетворяющих требованиям, и проверке существующих S-блоков

5.1 Поиск S-блоков, удовлетворяющих криптографическим требованиям

По итогам проведенных исследований, касающихся криптографических свойств нелинейного биективного преобразования, представленного в криптографических алгоритмах в виде подстановок S-блоков, с помощью разработанного программного обеспечения был произведен поиск S-блоков, удовлетворяющих заданным требованиям, используя соответствующий режим.

Для поиска S-блоков степени 16 в качестве начального заполнения мастер-ключа использовался текст «Penza_PGU_FPITE_IBST_diplom_2017», а вектора инициализации IV – «Penza_PG».

В результате поиска найдено несколько S-блоков. Восемь S-блоков, которые могли бы быть использованы в криптографическом алгоритме ГОСТ Р 28147-89 представлены на рисунке 39.

$$\begin{aligned}\pi_0(x) &= (7, 9, 0, 1, 14, 3, 12, 4, 10, 11, 6, 2, 5, 8, 15, 13) \\ \pi_1(x) &= (14, 15, 5, 4, 11, 9, 2, 13, 1, 10, 7, 8, 0, 12, 3, 6) \\ \pi_2(x) &= (3, 7, 10, 8, 2, 9, 15, 5, 11, 0, 1, 12, 14, 6, 13, 4) \\ \pi_3(x) &= (9, 4, 8, 5, 6, 0, 3, 10, 12, 2, 1, 15, 13, 11, 7, 14) \\ \pi_4(x) &= (6, 15, 1, 13, 9, 4, 12, 11, 10, 8, 14, 0, 3, 5, 2, 7) \\ \pi_5(x) &= (12, 15, 7, 14, 9, 13, 5, 10, 11, 2, 3, 0, 1, 8, 6, 4) \\ \pi_6(x) &= (6, 0, 15, 14, 5, 13, 9, 2, 4, 10, 11, 8, 7, 12, 1, 3) \\ \pi_7(x) &= (5, 14, 12, 11, 15, 3, 10, 2, 9, 13, 7, 4, 1, 6, 0, 8)\end{aligned}$$

Рисунок 39 – Найденный S-блок степени 16

Характеристики найденных S-блоков:

- S-блок π_0 :

а) отсутствие неподвижных точек: да;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 172;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

- S-блок π_1 :

а) отсутствие неподвижных точек: да;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 166;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

- S-блок π_2 :

- а) отсутствие неподвижных точек: да;
- б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;
- в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 167;
- г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;
- д) максимальная разностная характеристика координатных функций ниже или равна порогу: нет; Значение: 6;
- е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;
- ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;
- з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов: 1;

- S-блок π_3 :

- а) отсутствие неподвижных точек: да;
- б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;
- в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 174;
- г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;
- д) максимальная разностная характеристика координатных функций ниже или равна порогу: нет; Значение: 8;
- е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;
- ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

- S-блок π_4 :

а) отсутствие неподвижных точек: да;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 166;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

- S-блок π_5 :

а) отсутствие неподвижных точек: да;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 165;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: нет; Значение: 6;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

- S-блок π_6 :

а) отсутствие неподвижных точек: да;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 165;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: нет; Значение: 6;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

- S-блок π_7 :

а) отсутствие неподвижных точек: да;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 166;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: нет; Значение: 6;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 1536; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 16 ; Количество циклов:1;

Для поиска S-блоков степени 256 в качестве начального заполнения мастер-ключа использовался текст «2017_PGU_Penza_FPITE_IBST_diplom», а вектора инициализации IV – «2017_PGU».

В результате поиска найдено несколько S-блоков. Один из таких S-блоков представлен на рисунке 40.

```
201 61 73 146 50 149 69 166 204 4 25 192 118 49 3 93
161 37 55 198 53 239 187 11 82 182 84 32 117 242 228 127
165 62 170 119 208 105 74 20 83 91 59 179 18 246 205 107
145 193 28 181 211 80 168 66 134 150 125 163 10 58 15 56
154 48 9 128 17 232 87 30 158 234 72 13 249 39 114 184
212 63 200 81 206 86 16 209 218 151 219 71 167 157 138 99
85 98 243 253 27 237 112 42 64 126 21 19 141 24 250 0
34 135 88 1 226 255 217 46 5 90 123 190 172 60 225 79
186 36 175 156 68 67 177 76 45 132 121 100 251 44 210 133
216 207 235 197 97 142 191 139 189 194 223 31 104 148 22 115
2 95 227 78 102 171 214 164 43 221 70 236 241 230 233 231
140 57 137 113 38 178 185 153 51 240 94 109 215 155 116 199
147 162 65 101 52 130 106 196 254 122 144 247 23 120 108 222
12 129 220 26 203 7 14 169 33 152 35 176 6 224 89 111
131 77 238 8 136 245 244 252 40 229 47 160 248 183 174 54
143 213 202 41 188 195 29 75 96 92 103 110 124 159 173 180
```

Рисунок 40 – Найденный S-блок степени 256

Перед генерацией данного S-блока было выполнено 583088 холостых итераций.

Характеристики данного S-блока:

– отсутствие неподвижных точек: да;

- максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 7;
- число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 41205;
- минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 7;
- максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 10;
- минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 104;
- GAC: да; Max sigma: 132096; Max delta: 56;
- минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 145 89 22; Количество циклов: 3;

5.2 Проверка применяемых в криптографических алгоритмах S-блоков

В данном разделе проведено исследование S-блоков, применяемых в настоящее время в криптографических алгоритмах.

Для проведения данного исследования взяты S-блоки таких криптографических алгоритмов как ГОСТ Р 34.12-2015 (шифр «Кузнечик» и шифр «Магма»), AES.

В качестве нелинейного биективного преобразования в алгоритме блочного шифрования «Кузнечик» используется S-блок степени 256, представленный на рисунке 5.

С помощью разработанного программного обеспечения произведена проверка данного S-блока. Получены следующие результаты:

- отсутствие неподвижных точек: да;
- максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 7;

- число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 41224;
- минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 7;
- максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 8;
- минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 104;
- GAC: нет; Max sigma: 162816; Max delta: 64;
- минимальная длина циклов и их количество выше или равно порогу: да; Длины циклов: 243 13; Количество циклов: 2;

Как видно из результатов, данный S-блок не прошел проверку по глобальному лавинному критерию по, что означает наличие не очень хороших лавинных характеристик.

В алгоритме блочного шифрования AES, разработанном в США, также используется S-блок степени 256, который представлен на рисунке 8.

Результаты проверки данного S-блока:

- отсутствие неподвижных точек: да;
- максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 7;
- число ненулевых элементов матрицы частот выше порога или равно ему: да; Значение: 41239;
- минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 7;
- максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;
- минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 112;
- GAC: да; Max sigma: 67584; Max delta: 32;

— минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 59 81 87 27 2 ; Количество циклов:5;

Данный S-блок, по сравнению с S-блоком алгоритма «Кузнечик», обладает лучшими показателями по разностной характеристике и нелинейности, также он имеет отличные лавинные характеристики, но его циклическая структура не соответствует таким требованиям как минимальная длина цикла и их количество.

Алгоритм блочного шифрования «Магма» является аналогом ГОСТ 28147-89. Основным его отличием является зафиксированные значения S-блоков степени 16 в количестве 8 штук, представленных на рисунке 6.

Результаты проверки данных S-блоков:

- S-блок π_0 :

- а) отсутствие неподвижных точек: нет;
- б) максимальный элемент матрицы частот ниже порога или равно ему: нет; Значение: 4;
- в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 143;
- г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: нет; Значение: 2;
- д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;
- е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;
- ж) глобальный лавинный критерий: нет; Max sigma:768; Max delta: 16;
- з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 2 13 1 ; Количество циклов:3;

- S-блок π_1 :

- а) отсутствие неподвижных точек: нет;

б) максимальный элемент матрицы частот ниже порога или равно ему: нет; Значение: 4;

в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 140;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: нет; Значение: 2;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: нет; Max sigma: 768; Max delta: 16;

з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 7 2 1 1 3 1 1 ; Количество циклов: 7;

- S-блок π_2 :

а) отсутствие неподвижных точек: нет;

б) максимальный элемент матрицы частот ниже порога или равно ему: нет; Значение: 5;

в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 132;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: да; Max sigma: 384; Max delta: 8;

з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 6 9 1 ; Количество циклов: 3;

- S-блок π_3 :

- а) отсутствие неподвижных точек: нет;
- б) максимальный элемент матрицы частот ниже порога или равно ему: нет; Значение: 4;
- в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 145;
- г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;
- д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;
- е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;
- ж) глобальный лавинный критерий: да; Max sigma: 384; Max delta: 8;
- з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 14 1 1 ; Количество циклов: 3;

- S-блок π_4 :

- а) отсутствие неподвижных точек: нет;
- б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;
- в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 154;
- г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: нет; Значение: 2;
- д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;
- е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;
- ж) глобальный лавинный критерий: нет; Max sigma: 768; Max delta: 16;

з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 5 7 2 1 1 ; Количество циклов:5;

- S-блок π_5 :

а) отсутствие неподвижных точек: нет;

б) максимальный элемент матрицы частот ниже порога или равно ему: нет; Значение: 4;

в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 154;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: нет; Значение: 2;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

ж) глобальный лавинный критерий: нет; Max sigma:768;Max delta: 16;

з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 4 11 1 ; Количество циклов:3;

- S-блок π_6 :

а) отсутствие неподвижных точек: нет;

б) максимальный элемент матрицы частот ниже порога или равно ему: да; Значение: 3;

в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 156;

г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: нет; Значение: 2;

д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;

е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;

- ж) глобальный лавинный критерий: нет; Max sigma:768;Max delta: 16;
- з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 8 7 1 ; Количество циклов:3;
- S-блок π_7 :
 - а) отсутствие неподвижных точек: нет;
 - б) максимальный элемент матрицы частот ниже порога или равно ему: нет; Значение: 5;
 - в) число ненулевых элементов матрицы частот выше порога или равно ему: нет; Значение: 143;
 - г) минимальная степень полинома Жегалкина координатных функций выше или равна порогу: да; Значение: 3;
 - д) максимальная разностная характеристика координатных функций ниже или равна порогу: да; Значение: 4;
 - е) минимальное значение нелинейности координатных функций выше или равно порогу: да; Значение: 4;
 - ж) глобальный лавинный критерий: да; Max sigma: 384; Max delta: 8;
 - з) минимальная длина циклов и их количество выше или равно порогу: нет; Длины циклов: 14 1 1 ; Количество циклов:3.

Как видно из результатов проведенной проверки, все S-блоки шифра «Магма» прошли проверку на разностную характеристику и нелинейность, но полностью провалили проверку на лавинные критерии: и проверку на матрицу частот, и проверку на глобальный лавинный критерий. Также во всех представленных S-блоках присутствуют «фиксированные» точки, что говорит о слабых перемешивающих свойствах S-блоков. Обращает на себя внимание и свойства циклов, которые, ввиду наличия фиксированных точек, также имеют слабые характеристики по их количеству и длине.

Таким образом, после проведения исследования S-блоков одних из популярных алгоритмов шифрования сделан вывод о том, что наилучшие свойства S-блока у алгоритма блочного шифрования AES, немного по данным

свойствам уступает S-блок алгоритма «Кузнечик», а у шифра «Магма» - S-блоки плохого качества.

Зарубежным аналогом алгоритма «Магма» (он же ГОСТ 28147-89) является криптографический алгоритм DES.

Нелинейное биективное преобразование DES состоит из восьми групп подстановок S_i , каждая из которых представлена четырьмя S-блоками степени 16.

Данное преобразование представлено на рисунке 7.

Исследование нелинейного биективного преобразования проводилось над группами подстановок, результат которого представлен в таблицах 3-10.

Таблица 3 – Исследование группы подстановок S_1

Требования \ S-блоки	S_{10}	S_{11}	S_{12}	S_{13}
Отсутствие «неподвижных» точек	да	нет	нет	нет
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 5	да, значение: 3	нет, значение: 4	нет, значение: 6
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 140	нет, значение: 146	нет, значение: 150	нет, значение: 142
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	да, значение: 3	да, значение: 3	да, значение: 3	нет, значение: 2
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 8	нет, значение: 8	нет, значение: 8	нет, значение: 8
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	да, значение: 4	да, значение: 4	да, значение: 4	да, значение: 4
Глобальный лавинный критерий; значение параметра	да, значения: $\max \sigma = 384$ $\max \Delta = 8$	да, значения: $\max \sigma = 384$ $\max \Delta = 8$	да, значения: $\max \sigma = 384$ $\max \Delta = 8$	нет, значения: $\max \sigma = 768$ $\max \Delta = 16$

Продолжение таблицы 3

Требования \ S-блоки	S ₁₀	S ₁₁	S ₁₂	S ₁₃
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 2 14; количество: 2	нет, длины циклов: 1 11 3 1; количество: 4	нет, длины циклов: 9 1 4 2; количество: 4	нет, длины циклов: 3 11 1 1; количество: 4

Таблица 4 – Исследование группы подстановок S₂

Требования \ S-блоки	S ₂₀	S ₂₁	S ₂₂	S ₂₃
Отсутствие «неподвижных» точек	нет	да	нет	нет
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 6	нет, значение: 4	нет, значение: 5	нет, значение: 5
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 136	нет, значение: 152	нет, значение: 128	нет, значение: 144
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	нет, значение: 2	нет, значение: 2	нет, значение: 2	да, значение: 3
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 6	нет, значение: 8	нет, значение: 8	нет, значение: 6
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	да, значение: 4	нет, значение: 2	нет, значение: 2	да, значение: 4
Глобальный лавинный критерий; значение параметра	нет, значения: max σ = 1152 max Δ = 16	нет, значения: max σ = 1152 max Δ = 16	нет, значения: max σ = 1152 max Δ = 16	да, значения: max σ = 384 max Δ = 8
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 14 1 1; количество: 3	нет, длины циклов: 9 4 3; количество: 3	нет, длины циклов: 1 4 4 6 1; количество: 5	нет, длины циклов: 12 3 1; количество: 3

Таблица 5 – Исследование группы подстановок S_3

Требования \ S-блоки	S_{30}	S_{31}	S_{32}	S_{33}
Отсутствие «неподвижных» точек	да	нет	нет	нет
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 4	нет, значение: 5	нет, значение: 5	нет, значение: 4
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 152	нет, значение: 142	нет, значение: 152	нет, значение: 142
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	нет, значение: 2	нет, значение: 2	нет, значение: 2	нет, значение: 2
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 8	нет, значение: 8	нет, значение: 8	нет, значение: 8
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	да, значение: 4	нет, значение: 2	да, значение: 4	да, значение: 4
Глобальный лавинный критерий; значение параметра	нет, значения: $\max \sigma = 1152$ $\max \Delta = 16$	нет, значения: $\max \sigma = 1152$ $\max \Delta = 16$	нет, значения: $\max \sigma = 1152$ $\max \Delta = 16$	нет, значения: $\max \sigma = 1152$ $\max \Delta = 16$
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	да, длины циклов: 16; количество: 1	нет, длины циклов: 14 1 1; количество: 3	нет, длины циклов: 11 4 1; количество: 3	нет, длины циклов: 12 3 1; количество: 3

Таблица 6 – Исследование группы подстановок S_4

Требования \ S-блоки	S_{40}	S_{41}	S_{42}	S_{43}
Отсутствие «неподвижных» точек	нет	нет	да	нет
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 6	нет, значение: 4	нет, значение: 5	нет, значение: 5

Продолжение таблицы 6

Требования \ S-блоки	S ₄₀	S ₄₁	S ₄₂	S ₄₃
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 118	нет, значение: 142	нет, значение: 130	нет, значение: 112
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	да, значение: 3	да, значение: 3	да, значение: 3	да, значение: 3
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 6	нет, значение: 6	нет, значение: 6	нет, значение: 6
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	да, значение: 4	да, значение: 4	да, значение: 4	да, значение: 4
Глобальный лавинный критерий; значение параметра	да, значения: $\max \sigma = 384$ $\max \Delta = 8$	да, значения: $\max \sigma = 384$ $\max \Delta = 8$	да, значения: $\max \sigma = 384$ $\max \Delta = 8$	да, значения: $\max \sigma = 384$ $\max \Delta = 8$
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 14 1 1; количество: 3	нет, длины циклов: 10 5 1; количество: 3	нет, длины циклов: 3 6 7; количество: 3	нет, длины циклов: 14 1 1; количество: 3

Таблица 7 – Исследование группы подстановок S₅

Требования \ S-блоки	S ₅₀	S ₅₁	S ₅₂	S ₅₃
Отсутствие «неподвижных» точек	нет	нет	нет	да
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 4	нет, значение: 4	да, значение: 3	нет, значение: 4
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 154	нет, значение: 144	нет, значение: 154	нет, значение: 136

Продолжение таблицы 7

Требования \ S-блоки	S ₅₀	S ₅₁	S ₅₂	S ₅₃
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	нет, значение: 2	да, значение: 3	да, значение: 3	да, значение: 3
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 8	нет, значение: 6	нет, значение: 6	нет, значение: 6
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	нет, значение: 2	да, значение: 4	нет, значение: 2	да, значение: 4
Глобальный лавинный критерий; значение параметра	нет, значения: max σ = 1152 max Δ = 16	да, значения: max σ = 384 max Δ = 8	нет, значения: max σ = 1152 max Δ = 8	нет, значения: max σ = 1152 max Δ = 8
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 14 1 1; количество: 3	нет, длины циклов: 12 1 2 1; количество: 4	нет, длины циклов: 9 2 4 1; количество: 4	нет, длины циклов: 14 2; количество: 2

Таблица 8 – Исследование группы подстановок S₆

Требования \ S-блоки	S ₆₀	S ₆₁	S ₆₂	S ₆₃
Отсутствие «неподвижных» точек	нет	да	нет	да
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	да, значение: 3	нет, значение: 7	нет, значение: 4	нет, значение: 5
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 156	нет, значение: 126	нет, значение: 142	нет, значение: 150
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	да, значение: 3	да, значение: 3	да, значение: 3	нет, значение: 2

Продолжение таблицы 8

Требования \ S-блоки	S ₆₀	S ₆₁	S ₆₂	S ₆₃
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	да, значение: 4	нет, значение: 8	нет, значение: 6	нет, значение: 6
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	да, значение: 4	да, значение: 4	нет, значение: 2	да, значение: 4
Глобальный лавинный критерий; значение параметра	да, значения: max σ = 384 max Δ = 8	да, значения: max σ = 384 max Δ = 8	нет, значения: max σ = 1152 max Δ = 8	нет, значения: max σ = 1152 max Δ = 16
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 14 1 1; количество: 3	нет, длины циклов: 11 5; количество: 2	нет, длины циклов: 2 9 4 1; количество: 4	нет, длины циклов: 14 1 1; количество: 3

Таблица 9 – Исследование группы подстановок S₇

Требования \ S-блоки	S ₇₀	S ₇₁	S ₇₂	S ₇₃
Отсутствие «неподвижных» точек	нет	нет	да	нет
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 4	нет, значение: 4	нет, значение: 4	нет, значение: 4
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 158	нет, значение: 148	нет, значение: 154	нет, значение: 154
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	нет, значение: 2	нет, значение: 2	да, значение: 3	да, значение: 3
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 8	нет, значение: 8	нет, значение: 6	нет, значение: 8
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	нет, значение: 2	нет, значение: 2	да, значение: 4	нет, значение: 2

Продолжение таблицы 9

Требования \ S-блоки	S ₇₀	S ₇₁	S ₇₂	S ₇₃
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	нет, значение: 2	нет, значение: 2	да, значение: 3	да, значение: 3
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 8	нет, значение: 8	нет, значение: 6	нет, значение: 8
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	нет, значение: 2	нет, значение: 2	да, значение: 4	нет, значение: 2
Глобальный лавинный критерий; значение параметра	нет, значения: max σ = 1152 max Δ = 16	нет, значения: max σ = 1152 max Δ = 16	да, значения: max σ = 384 max Δ = 8	нет, значения: max σ = 1152 max Δ = 8
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 11 1 4; количество: 3	нет, длины циклов: 5 3 5 1 2; количество: 5	нет, длины циклов: 4 9 3; количество: 3	нет, длины циклов: 3 10 2 1; количество: 4

Таблица 10 – Исследование группы подстановок S₈

Требования \ S-блоки	S ₈₀	S ₈₁	S ₈₂	S ₈₃
Отсутствие «неподвижных» точек	нет	нет	нет	нет
Максимальный элемент матрицы частот ниже порога или равно ему; значение параметра	нет, значение: 4	нет, значение: 4	нет, значение: 4	нет, значение: 4
Число ненулевых элементов матрицы частот выше порога или равно ему; значение параметра	нет, значение: 138	нет, значение: 156	нет, значение: 132	нет, значение: 152
Минимальная степень полинома Жегалкина координатных функций выше или равна порогу; значение параметра	нет, значение: 2	да, значение: 3	да, значение: 3	нет, значение: 2

Продолжение таблицы 10

Требования \ S-блоки	S ₈₀	S ₈₁	S ₈₂	S ₈₃
Максимальная разностная характеристика координатных функций ниже или равна порогу; значение параметра	нет, значение: 6	нет, значение: 10	нет, значение: 6	нет, значение: 8
Минимальное значение нелинейности координатных функций выше или равно порогу; значение параметра	да, значение: 4	нет, значение: 2	да, значение: 4	да, значение: 4
Глобальный лавинный критерий; значение параметра	нет, значения: max σ = 1152 max Δ = 16	нет, значения: max σ = 1152 max Δ = 8	да, значения: max σ = 1152 max Δ = 8	нет, значения: max σ = 768 max Δ = 16
Минимальная длина циклов и их количество выше или равно порогу; значение параметра	нет, длины циклов: 2 13 1; количество: 3	нет, длины циклов: 11 4 1; количество: 3	нет, длины циклов: 11 4 1; количество: 3	нет, длины циклов: 7 1 7 1; количество: 4

По результатам исследования нелинейного биективного преобразования DES определено, что данная комбинация S-блоков не соответствует определенным ранее критериям. Таким образом можно сделать вывод о том, что нелинейное биективное преобразование не отвечает требованиям криптографической стойкости.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта был проведен обзор применения нелинейного биективного преобразования в криптографических алгоритмах; определены характеристики, необходимые для обеспечения криптостойкости нелинейного биективного преобразования, такие как:

- максимальный элемент матрицы встречаемости разностей переходов ненулевых биграмм $P(\pi)$;
- число ненулевых элементов матрицы $P(\pi)$;
- минимальная степень нелинейности N_f координатных функций;
- минимальная алгебраическая степень координатных функций;
- максимальная разностная характеристика координатных функций;
- минимальная длина циклов, представляющих S-блок;
- максимальное количество циклов;
- отсутствие неподвижных точек подстановки;
- максимальная сумма квадратов значений автокорреляционной функции по всем направлениям u ;
- максимальное значение автокорреляционной функции по всем направлениям u , кроме $u \neq 0$.

Также разработан алгоритм генерации узлов замены в соответствии с требованиями «доказуемой» псевдослучайности и методика и рекомендации по работе с разработанным программным обеспечением.

Разработанное программное обеспечение осуществляет работу в следующих режимах:

- проверка выбранного нелинейного биективного преобразования на соответствие заданным требованиям;
- поиск подмножества нелинейных биективных преобразований, соответствующих заданным требованиям;

Генерация узлов замены осуществлена в соответствии с требованиями «доказуемой» псевдослучайности. В качестве базового криптографического

механизма использован ГОСТ Р 34.12-2015 («Кузнечик») в режиме гаммирования (ГОСТ Р 34.13-2015). Разработанное программное обеспечение имеет графический интерфейс и функционирует на ОС семейства Windows версии не ниже Windows 7.

Таким образом, задание на дипломный проект выполнено в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Доктрина информационной безопасности Российской Федерации [Электронный ресурс]. – Режим доступа: <http://static.kremlin.ru/media/acts/files/0001201612060002.pdf>.
2. ГОСТ Р 34.12–2015 «Информационная технология. Криптографическая защита информации. Блочные шифры» [Электронный ресурс]. – Режим доступа: http://www.tc26.ru/standard/gost/GOST_R_3412-2015.pdf.
3. ГОСТ 28147-89 «Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования» [Электронный ресурс]. – Режим доступа: <http://gostrf.com/normadata/1/4294826/4294826631.pdf>.
4. ГОСТ Р 34.13–2015 «Информационная технология. Криптографическая защита информации. Режимы работы блочных шифров» [Электронный ресурс]. – Режим доступа: http://www.tc26.ru/standard/gost/GOST_R_3413-2015.pdf.
5. Криптографическая стойкость [Электронный ресурс]. – Режим доступа: https://ru.wikipedia.org/wiki/Криптографическая_стойкость.
6. К. Шеннон «Теория связи в секретных системах» [Электронный ресурс]. – Режим доступа: <http://kek.ksu.ru/student/crypto/files/shannon.pdf>.
7. Б. Шнайер «Прикладная криптография». – Режим доступа: <http://www.studfiles.ru/preview/951064/>.
8. Data Encryption Standard [Электронный ресурс]. – Режим доступа: <https://ru.wikipedia.org/wiki/DES>.
9. Advanced Encryption Standard [Электронный ресурс]. – Режим доступа: https://ru.wikipedia.org/wiki/Advanced_Encryption_Standard.

10. М. Е. Масленников «Криптография и свобода». – Режим доступа: <http://bookzz.org/dl/584784/19c021>.
11. Г. П. Агibalов, И. А. Панкратова «Элементы теории статистических аналогов дискретных функций с применением в криптоанализе и итеративных блочных шифров» [Электронный ресурс].- Режим доступа: <http://www.mathnet.ru/links/aac3f8ec49ac29d393a48d4aae31763e/pdm240.pdf>.
12. М. Б. Бровкова, М. О. Оболонский «Применение дифференциального и линейного криптоалгоритмов для анализа стойкости криптографических систем» [Электронный ресурс]. – Режим доступа: <http://cyberleninka.ru/article/n/primenenie-differentsialnogo-i-lineynogo-kriptoalgoritmov-dlya-analiza-stoykosti-kriptograficheskikh-sistem.pdf>.
13. А. А. Городилова «От криптоанализа шифра к криптографическому свойству булевой функции» [Электронный ресурс]. - Режим доступа: <http://www.mathnet.ru/links/8bd641c1aeef76f6f66183180bca1adf/pdm552.pdf>.
14. K. Nyberg «Perfect nonlinear S-boxes» [Электронный ресурс]. - Режим доступа: https://link.springer.com/chapter/10.1007%2F3-540-46416-6_32.
15. Р. В. Сергиенко, И. В. Московченко «Исследование криптографических свойств нелинейных узлов замен алгоритма симметричного шифрования ГОСТ 28147-89». – Режим доступа: http://www.hups.mil.gov.ua/periodic-app/article/5898/soi_2007_8_26.pdf.
16. P. Tesar «A new method for generating high non-linearity S-boxes» [Электронный ресурс]. – Режим доступа: http://www.radioeng.cz/fulltexts/2010/10_01_023_026.pdf.
17. N. T. Courtois, J. Pieprzyk «Cryptanalysis of block ciphers with overdefined systems of equations». – Режим доступа: <https://eprint.iacr.org/2002/044.pdf>.
18. Полином Жегалкина [Электронный ресурс]. – Режим доступа: https://ru.wikipedia.org/wiki/Полином_Жегалкина.

19. G. Ivanov, N. Nikolov, S. Nikova «Cryptographically strong S-boxes generated by modified immune Algorithm» [Электронный ресурс]. – Режим доступа: <https://conferences.matheo.si/event/16/contribution/21/material/paper/0.pdf>.
20. L. Budaghyan, A. Pott «On differential uniformity and nonlinearity of functions» [Электронный ресурс]. – Режим доступа: http://ac.els-cdn.com/S0012365X07010102/1-s2.0-S0012365X07010102-main.pdf?_tid=5a80d1a0-38d2-11e7-b4bd-00000aacb360&acdnat=1494786357_976f51e24a13ae2791aa5e0d0c5cf002.
21. Ю. Н. Зайко «Криптография глазами физика» [Электронный ресурс]. – Режим доступа: http://optics.sgu.ru/SFM_Files/2008/report/549/#_ftn1.
22. R. Forre «The strict avalanche criterion: spectral properties of Boolean functions and an extended definition» [Электронный ресурс]. – Режим доступа: http://download.springer.com/static/pdf/980/chp%253A10.1007%252F0-387-34799-2_31.pdf?originUrl=http%3A%2F%2Flink.springer.com%2Fchapter%2F10.1007%2F0-387-34799-2_31&token2=exp=1494789017~acl=%2Fstatic%2Fpdf%2F980%2Fchp%25253A10.1007%25252F0-387-34799-2_31.pdf%3ForiginUrl%3Dhttp%253A%252F%252Flink.springer.com%252Fchapter%252F10.1007%252F0-387-34799-2_31*~hmac=0f754e73f278ce8e35e809fd42eec63f39fb479f405d84db772c1225cd38e9b5.
23. Xian-Mo Zhang, Y. Zheng «GAC – the criterion for Global Avalanche Characteristics of cryptographic functions» [Электронный ресурс]. – Режим доступа: http://www.jucs.org/jucs_1_5/gac_the_criterion_for/Zhang_X.pdf.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программного обеспечения для построения криптостойкого
нелинейного биективного преобразования

Листинг А.1 – Исходный код файла Kuznechik.cpp

```
#include <iostream>
#include <cstring>
#include "Kuznechik.h"
using namespace std;
void CTR_crypt(int *sbox, int n, int *master_key, int *IV_CTR){
    //s = 128
    static int pi[256] = {
0xFC, 0xEE, 0xDD, 0x11, 0xCF, 0x6E, 0x31, 0x16, 0xFB, 0xC4, 0xFA, 0xDA, 0x23,
0xC5, 0x04, 0x4D,
0xE9, 0x77, 0xF0, 0xDB, 0x93, 0x2E, 0x99, 0xBA, 0x17, 0x36, 0xF1, 0xBB, 0x14,
0xCD, 0x5F, 0xC1,
0xF9, 0x18, 0x65, 0x5A, 0xE2, 0x5C, 0xEF, 0x21, 0x81, 0x1C, 0x3C, 0x42, 0x8B,
0x01, 0x8E, 0x4F,
0x05, 0x84, 0x02, 0xAE, 0xE3, 0x6A, 0x8F, 0xA0, 0x06, 0x0B, 0xED, 0x98,
0x7F, 0xD4, 0xD3, 0x1F,
0xEB, 0x34, 0x2C, 0x51, 0xEA, 0xC8, 0x48, 0xAB, 0xF2, 0x2A, 0x68, 0xA2,
0xFD, 0x3A, 0xCE, 0xCC,
0xB5, 0x70, 0x0E, 0x56, 0x08, 0x0C, 0x76, 0x12, 0xBF, 0x72, 0x13, 0x47,
0x9C, 0xB7, 0x5D, 0x87,
0x15, 0xA1, 0x96, 0x29, 0x10, 0x7B, 0x9A, 0xC7, 0xF3, 0x91, 0x78, 0x6F,
0x9D, 0x9E, 0xB2, 0xB1,
0x32, 0x75, 0x19, 0x3D, 0xFF, 0x35, 0x8A, 0x7E, 0x6D, 0x54, 0xC6, 0x80,
0xC3, 0xBD, 0x0D, 0x57,
0xDF, 0xF5, 0x24, 0xA9, 0x3E, 0xA8, 0x43, 0xC9, 0xD7, 0x79, 0xD6, 0xF6,
0x7C, 0x22, 0xB9, 0x03,
0xE0, 0x0F, 0xEC, 0xDE, 0x7A, 0x94, 0xB0, 0xBC, 0xDC, 0xE8, 0x28, 0x50,
0x4E, 0x33, 0x0A, 0x4A,
0xA7, 0x97, 0x60, 0x73, 0x1E, 0x00, 0x62, 0x44, 0x1A, 0xB8, 0x38, 0x82,
0x64, 0x9F, 0x26, 0x41,
0xAD, 0x45, 0x46, 0x92, 0x27, 0x5E, 0x55, 0x2F, 0x8C, 0xA3, 0xA5, 0x7D,
0x69, 0xD5, 0x95, 0x3B,
0x07, 0x58, 0xB3, 0x40, 0x86, 0xAC, 0x1D, 0xF7, 0x30, 0x37, 0x6B, 0xE4,
0x88, 0xD9, 0xE7, 0x89,
0xE1, 0x1B, 0x83, 0x49, 0x4C, 0x3F, 0xF8, 0xFE, 0x8D, 0x53, 0xAA, 0x90,
0xCA, 0xD8, 0x85, 0x61,
0x20, 0x71, 0x67, 0xA4, 0x2D, 0x2B, 0x09, 0x5B, 0xCB, 0x9B, 0x25, 0xD0,
0xBE, 0xE5, 0x6C, 0x52,
0x59, 0xA6, 0x74, 0xD2, 0xE6, 0xF4, 0xB4, 0xC0, 0xD1, 0x66, 0xAF, 0xC2,
0x39, 0x4B, 0x63, 0xB6
    }; // таблица pi подстановки в режиме зашифрования
    int k[160]; // массив ключа
    int c[512] = { // в каждой строке c записаны числа i в 16сс виде
0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x03, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x05, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x06, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x07, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    };
```

```

0x09, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0b, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0d, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x11, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x12, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x13, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x14, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x15, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x16, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x17, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x19, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x1a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x1b, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x1c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x1d, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x1e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x1f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00 };
int b[16] = {0}; // шифртекст
int number1 [16] = {1, 0, 0, 0, 0, 0, 0, 0,
                    0, 0, 0, 0, 0, 0, 0, 0};
for (int i = 0; i < 16; i++) {
    k[i] = master_key[i + 16];
    k[i + 16] = master_key[i];
} // создание из мастер-ключа ключей k1 и k2
gen_c(c); // Генерация итерационных переменных
//cout_c(c); // Вывод итерационных переменных
gen_key(c, k, pi); // Генерация ключей
//int IV1[16], buf[16];
int l = 0;
memcpy(b, IV_CTR, 64);

```

```

for (int i = 0; i < n; i++){
    crypt(k, IV_CTR, b, pi);
    if (n == 16)
        sbbox[i] = b[0] % 0x10;
    else
        //sbox[i] = ((b[1] << 8) ^ b[0]) % 0x100;
        sbbox[i] = b[0] % 0x100;
    //проверка на одинаковые значения
    int buf = i;
    int m = 0;
    while(m < buf){
        if (sbox[m] == sbox[buf]){
            sbox[buf] = (sbox[buf] + 7) % n; // +2 если совпадают
            m = -1; }
        m++;
    }
    for (int count = 0; count < 16; count++){
        IV_CTR[count] += number1[count]; // суммируем байты чисел
        IV_CTR[count + 1] += (IV_CTR[count] / 256); // если есть разряд
для переноса, переносим его в следующий разряд
        IV_CTR[count] %= 256; // если есть разряд для переноса он
отсекается
    }
}
for (int i = 0; i < 16; i++) {
    master_key[i + 16] = k[i + 127];
    master_key[i] = k[i + 16 + 127];
} // переписывание мастер-ключа
}
void s(int *x, int *pi) { // преобразование S для маскирования
    for (int i = 0; i < 16; i++)
        x[i] = pi[x[i]];
}
void l(int *x) { // преобразование L
    for (int i = 0; i < 16; i++)
        r(x);
}
void r(int *x) { //преобразование R
    unsigned char temp = LFSR(x);
    for (int i = 0; i < 15; i++)
        x[i] = x[i + 1];
    x[15] = temp;
}
int LFSR(int *x) { // линейный сдвиг
    int q[16] = { 0x01, 0x94, 0x20, 0x85,
                  0x10, 0xc2, 0xc0, 0x01,
                  0xfb, 0x01, 0xc0, 0xc2,
                  0x10, 0x85, 0x20, 0x94 };
    multiple_gf(q, x); // умножение
    int znach = 0;
    for (int i = 0; i < 16; i++) // умножение полученных произведений по mod2
        znach ^= q[i];
    return znach;
}
void multiple_gf(int *q, int *x) {
    short buf1[16];
    short p = 451; //порождающий многочлен 111000011
    for (int i = 0; i < 16; i++) // умножение
        buf1[i] = q[i];
    for (int i = 0; i < 16; i++) {
        short res = 0;
        for (int j = 0; j < 8; j++) {

```

```

        if ((x[i] >> j) & 1 == 1)
            res ^= (buf1[i] << j);
    }
    buf1[i] = res;
}
for (int i = 0; i < 16; i++) { // деление на порождающий многочлен
    int k = razryad(buf1[i]);
    while (k >= 9) { // 9 - число разрядов в p
        int j = k - 9;
        buf1[i] ^= (p << j);
        k = razryad(buf1[i]);
    }
    q[i] = buf1[i];
}
}
int razryad(short v) {
    int z = 0; // счетчик разрядов
    while (v != 1) {
        if (v == 0) break;
        v /= 2;
        z++;
    }
    z++;
    return z;
}
void x_fun(int *x1, int *x2) { // преобразование X
    for (int i = 0; i < 16; i++)
        x1[i] = x2[i] ^ x1[i];
}
void gen_c(int *c) { // Генерация итерационных переменных
    int buf[16];
    for (int i = 0; i < 32; i++) {
        for (int j = 0; j < 16; j++)
            buf[j] = c[i * 16 + j];
        l(buf);
        for (int j = 0; j < 16; j++)
            c[i * 16 + j] = buf[j];
    }
}
void gen_key(int *c, int *k, int *pi) { // Генерация ключей
    network(c, k, 0, 0, pi); // для создания ключей k3, k4
    network(c, k, 8, 1, pi); // для создания ключей k5, k6
    network(c, k, 16, 2, pi); // для создания ключей k7, k8
    network(c, k, 24, 3, pi); // для создания ключей k9, k10
}
void network(int *c, int *k, int f, int g, int *pi) { // Сеть Фейстеля
    int buf1[16], buf2[16], buf3[16];
    for (int i = 0; i < 16; i++) {
        buf1[i] = k[32 * g + i];
        buf2[i] = k[32 * g + 16 + i];
    }
    for (int i = f; i < (f + 8); i++) {
        for (int j = 0; j < 16; j++)
            buf3[j] = c[16 * i + j];
        x_fun(buf3, buf1);
        s(buf3, pi);
        l(buf3);
        for (int j = 0; j < 16; j++)
            buf3[j] = buf3[j] ^ buf2[j];
        memcpy(buf2, buf1, 64);
        memcpy(buf1, buf3, 64);
    }
}

```

```

    }
    for (int i = 0; i < 16; i++) {
        k[32 * (g + 1) + i] = buf1[i];
        k[32 * (g + 1) + 16 + i] = buf2[i];
    }
}

void crypt(int *k, int *a, int *b, int *pi) { //Зашифрование
    int buf[16]; // для ключа
    memcpy(b, a, 64);
    for (int i = 0; i < 10; i++) {
        for (int j = 0; j < 16; j++)
            buf[j] = k[i * 16 + j];
        x_fun(b, buf);
        if (i != 9) {
            s(b, pi);
            l(b);
        }
    }
}

void cout_c(int *c) { // Вывод итерационных переменных
    cout << "Итерационные переменные: " << endl;
    for (int i = 0; i < 512; i++) {
        printf("%0.2x ", c[i]);
        if ((i + 1) % 16 == 0)
            cout << endl;
    }
    cout << endl;
}

void cout_key(int *k) { // Вывод ключей
    cout << "Ключи: " << endl;
    for (int i = 0; i < 160; i++) {
        printf("%0.2x ", k[i]);
        if ((i + 1) % 16 == 0)
            cout << endl;
    }
    cout << endl;
}

void sdvig(int *x) {
    int temp = x[15];
    for (int i = 14; i >= 0; i--)
        x[i + 1] = x[i];
    x[0] = temp;
}

```

Листинг А.2 – Исходный код файла mainwindow.cpp

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <sbox.h>
using namespace std;
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    Menu1_Submenu1();
    menuBar()->connect(ui->action_1, SIGNAL(triggered(bool)), this,
    SLOT(Menu1_Submenu1()));
    menuBar()->connect(ui->action_2, SIGNAL(triggered(bool)), this,
    SLOT(Menu1_Submenu2()));
}

void MainWindow::Menu1_Submenu1() {
    QWidget *widget = new QWidget();
    QGridLayout *layout = new QGridLayout(widget);

```

```

search_box = new QPushButton(tr("Начать поиск"));
clear_log = new QPushButton(tr("Удалить историю поиска"));
address_save = new QLineEdit();
save_box = new QPushButton(tr("Выбор пути"));
state_master_key_kuznechik = new QLineEdit();
state_IV= new QLineEdit();
size_box = new QComboBox();
cancel_model = new QPushButton(tr("Отмена"));
address_save->setReadOnly(true);
layout->addWidget(new QLabel (tr("Поиск S-блоков, удовлетворяющих
требованиям")), 0, 0, 1, 2);
layout->addWidget(new QLabel (tr("Путь сохранения S-блоков:")),1,0,1,1);
layout->addWidget(address_save, 1, 1, 1, 1);
layout->addWidget(save_box, 1, 2, 1, 1);
layout->addWidget(new QLabel (tr("Начальное заполнение мастер-ключа:")),
2, 0, 1, 2);
layout->addWidget(state_master_key_kuznechik, 3, 0, 1, 1);
layout->addWidget(new QLabel (tr("Начальное заполнение вектора
инициализации:")), 4, 0, 1, 2);
layout->addWidget(state_IV, 5, 0, 1, 1);
layout->addWidget(new QLabel (tr("Степень S-блока(n):")), 2, 1, 1, 1,
Qt::AlignRight);
layout->addWidget(size_box, 2, 2, 1, 1);
layout->addWidget(search_box, 6, 2, 1, 1);
layout->addWidget(cancel_model, 6, 2, 1, 1, Qt::AlignRight);
layout->addWidget(clear_log, 6, 1, 1, 1, Qt::AlignRight);
search_label = new QLabel (tr("Проверено: "));
find_label = new QLabel (tr("Найдено: "));
search_number = new QLabel ();
find_number = new QLabel ();
layout->addWidget(search_label, 3, 1, 1, 1, Qt::AlignRight);
layout->addWidget(find_label, 4, 1, 1, 1, Qt::AlignRight);
layout->addWidget(search_number, 3, 2, 1, 1, Qt::AlignRight);
layout->addWidget(find_number, 4, 2, 1, 1, Qt::AlignRight);
search_label->hide();
find_label->hide();
search_number->hide();
find_number->hide();
cancel_model->hide();
setCentralWidget(widget);
state_master_key_kuznechik->setMaxLength(32); //максимальное число
элементов в строке
state_IV->setMaxLength(8); //максимальное число элементов в строке
size_box->addItem("16", 0);
size_box->addItem("256", 0);
size_box->setCurrentIndex(-1);
connect(size_box, SIGNAL(currentIndexChanged(QString)), this,
SLOT(define_size(QString))); // определение размера S-блока
connect(save_box, SIGNAL(clicked(bool)), this,
SLOT(save_box_clicked()));
connect(search_box,SIGNAL(clicked(bool)),this,
SLOT(search_box_clicked()));
connect(clear_log,SIGNAL(clicked(bool)),this,
SLOT(clear_log_clicked()));
widget->setAttribute(Qt::WA_DeleteOnClose);
}
void MainWindow::clear_log_clicked(){
    QFile json("config\\log.json");
    json.open(QIODevice::ReadOnly);
    QJsonParseError jerror;
    QJsonDocument jdoc= QJsonDocument::fromJson(json.readAll(),&jerror);

```

```

        json.close();
        if (jerror.error != QJsonParseError::NoError) {
            QMessageBox::warning(0, "Файл поврежден!", "Поврежден log-файл!");
            return;
        }
        QJsonArray master_key_start_json =
jdoc.object()["master_key_start"].toArray();
        QJsonArray IV_CTR_start_json = jdoc.object()["IV_CTR_start"].toArray();
        QJsonArray count_search_json = jdoc.object()["count_search"].toArray();
        QJsonArray count_sbox_json = jdoc.object()["count_sbox"].toArray();
        QJsonArray sizeofbox_json = jdoc.object()["sizeofbox"].toArray();
        QJsonArray master_key_current_json =
jdoc.object()["master_key_current"].toArray();
        QJsonArray IV_CTR_current_json = jdoc.object()["IV_CTR_current"].toArray();
        if ((master_key_start_json.size() != 0) & (IV_CTR_start_json.size() != 0) &
(sizeofbox_json.size() != 0)) {
            QMessageBox *msgBox = new QMessageBox;
            msgBox->setWindowTitle("Внимание!");
            msgBox->setText("Вы уверены?");
            msgBox->setStandardButtons(QMessageBox::Yes | QMessageBox::No);
            msgBox->setButtonText(QMessageBox::Yes, "Да");
            msgBox->setButtonText(QMessageBox::No, "Нет");
            msgBox->setDefaultButton(QMessageBox::No);
            int ret = msgBox->exec(); // возврат значения нажатой кнопки
            delete msgBox;
            if (ret == QMessageBox::Yes) {
                int count = master_key_start_json.size();
                for (int i = 0; i < count; i++) {
                    master_key_start_json.removeAt(0);
                    IV_CTR_start_json.removeAt(0);
                    count_search_json.removeAt(0);
                    count_sbox_json.removeAt(0);
                    sizeofbox_json.removeAt(0);
                    master_key_current_json.removeAt(0);
                    IV_CTR_current_json.removeAt(0);
                }
                QJsonObject myJsonObj;
                myJsonObj["master_key_start"] = master_key_start_json;
                myJsonObj["IV_CTR_start"] = IV_CTR_start_json;
                myJsonObj["count_search"] = count_search_json;
                myJsonObj["count_sbox"] = count_sbox_json;
                myJsonObj["sizeofbox"] = sizeofbox_json;
                myJsonObj["master_key_current"] = master_key_current_json;
                myJsonObj["IV_CTR_current"] = IV_CTR_current_json;
                QJsonDocument doc(myJsonObj);
                json.open(QIODevice::WriteOnly | QIODevice::Text);
                json.write(doc.toJson(QJsonDocument::Indented));
                json.close();
                QMessageBox::information(0, "Внимание!", "История очищена!");
            }
            if (ret == QMessageBox::No)
                return;
        }
        else
            QMessageBox::warning(0, "Внимание!", "История пуста!");
    }

void MainWindow::search_box_clicked() {
    if ((address_save->text() == "")) {
        QMessageBox::warning(0, "Ошибка ввода", "Не указан путь сохранения!");
        return;
    }
}

```

```

        if ((state_master_key_kuznechik->text().length() < 32) | (state_IV->
text().length() < 8)) {
            QMessageBox::warning(0, "Ошибка ввода", "Начальные значения слишком
короткие!");
            return;
        }
        if ((state_master_key_kuznechik->text().size() !=
state_master_key_kuznechik->text().toUtf8().size()) | (state_IV->text().size()
!= state_IV->text().toUtf8().size())) {
            QMessageBox::warning(0, "Ошибка ввода", "Введены кириллические
символы!");
            return;
        }
        if ((n != 16) & (n != 256)) {
            QMessageBox::warning(0, "Ошибка ввода", "Не выбран размер S-блока!");
            return;
        }
        QString master_key_string = state_master_key_kuznechik->text();
        QString IV_CTR_string = state_IV->text();
        //получение массива мастер-ключа и синхропосылки из введенных данных
        int *master_key = new int [master_key_string.length()];
        int *IV_CTR = new int [IV_CTR_string.length() + 8];
        for (int i = 0; i < 16; i++)
            IV_CTR[i] = 0;
        for (int i = 0; i < master_key_string.length(); i++) {
            master_key[i] = master_key_string[i].toLatin1();
        }
        for (int i = 0; i < IV_CTR_string.length(); i++) {
            IV_CTR[IV_CTR_string.length()+i]=IV_CTR_string[i].toLatin1();
        }
        QString search = "", find = "";
        int count_sbox = 0, count_search = 0;
        QString save_file = address_save->text();
        QFile file(save_file);
        QTextStream writeStream(&file);
        file.open(QIODevice::WriteOnly);
        QFile json("config\\log.json");
        json.open(QIODevice::ReadOnly);
        QJsonParseError jerror;
        QJsonDocument jdoc= QJsonDocument::fromJson(json.readAll(), &jerror);
        json.close();
        if(jerror.error != QJsonParseError::NoError){
            QMessageBox::warning(0, "Файл поврежден!", "Поврежден log-файл!");
            return;
        }
        QJsonArray master_key_start_json =
jdoc.object()["master_key_start"].toArray();
        QJsonArray IV_CTR_start_json = jdoc.object()["IV_CTR_start"].toArray();
        QJsonArray count_search_json = jdoc.object()["count_search"].toArray();
        QJsonArray count_sbox_json = jdoc.object()["count_sbox"].toArray();
        QJsonArray sizeofbox_json = jdoc.object()["sizeofbox"].toArray();
        QJsonArray master_key_current_json =
jdoc.object()["master_key_current"].toArray();
        QJsonArray IV_CTR_current_json = jdoc.object()["IV_CTR_current"].toArray();
        //int poisk = 0;
        if ((master_key_start_json.size() != 0) & (IV_CTR_start_json.size() != 0) &
(sizeofbox_json.size() != 0)) {
            for (int i = 0; i < master_key_start_json.size(); i++)
                if ((master_key_string == master_key_start_json[i].toString()) &
(IV_CTR_string == IV_CTR_start_json[i].toString()) & (n ==
sizeofbox_json[i].toInt())) {

```



```

        QString buf1, buf2, buf3;
        QString state = "Поиск S-блоков степени " +
buf1.setNum(sizeofbox_json[i].toInt()) + " при данных начальных состояниях
проводился ранее и приостановлен на итерации " +
buf2.setNum(count_search_json[i].toInt()) + ". Найдено: " +
buf3.setNum(count_sbox_json[i].toInt()) + ".\nВыберите действие";
        QMessageBox *msgBox = new QMessageBox;
        msgBox->setWindowTitle("Внимание!");
        msgBox->setText(state);
        msgBox->setStandardButtons(QMessageBox::Yes | QMessageBox::No |
QMessageBox::Cancel);
        msgBox->setDefaultButton(QMessageBox::Cancel);
        msgBox->setButtonText(QMessageBox::Yes, "Продолжить поиск
далее");

        msgBox->setButtonText(QMessageBox::No, "Начать поиск заново");
        msgBox->setButtonText(QMessageBox::Cancel, "Отменить поиск");
        int ret = msgBox->exec(); // возврат значения нажатой кнопки
        delete msgBox;
        if (ret == QMessageBox::Yes){
            count_search = count_search_json[i].toInt();
            count_sbox = count_sbox_json[i].toInt();
            QString master_key_current_json_string =
master_key_current_json[i].toString();
            QStringList master_key_current_list =
master_key_current_json_string.split(" ");
            for (int count = 0; count < master_key_current_list.size();
count++)
                master_key[count] =
master_key_current_list.at(count).toInt();
            QString IV_CTR_current_json_string =
IV_CTR_current_json[i].toString();
            QStringList IV_CTR_current_list =
IV_CTR_current_json_string.split(" ");
            for (int count = 0; count < IV_CTR_current_list.size();
count++)
                IV_CTR[count] = IV_CTR_current_list.at(count).toInt();
        }
        if (ret == QMessageBox::No){
            count_search = 0;
            count_sbox = 0;
        }
        if (ret == QMessageBox::Cancel)
            return;
        break;
    }

    }

    int *sbox = new int [n]; // объявление массива sbox
    //задание некликабельных элементов при проверке
    clear_log->hide();
    search_box->hide();
    cancel_model->show();
    search_label->show();
    find_label->show();
    search_number->show();
    find_number->show();
    save_box->setDisabled(true);
    state_master_key_kuznechik->setReadOnly(true);
    state_IV->setReadOnly(true);
    size_box->setDisabled(true);
    menuBar()->setDisabled(true);

```

```

//бесконечный поиск до нажатия клавиши "Отмена"
while (cancel_model->isDown() == false){
    QApplication::processEvents();
    QString ch1 = "", ch2 = "", ch3 = "", ch4 = "", ch5 = "", ch6 = "", ch7
= "", ch8 = "", length_cycle = "";
    int P2 = 0, P3 = 0, P4 = 0, P5 = 0, P6 = 0, delta = 0, count_cycles = 0;
    double sigma = 0;
    block_sbox_model(sbox, master_key, IV_CTR, n, ch1, ch2, ch3, ch4, ch5,
ch6, ch7, ch8, P2, P3, P4, P5, P6, sigma, delta, length_cycle, count_cycles); //
P_i - значение проверки
    //изменить условие
    if ((ch1 == "да") & (ch2 == "да") & (ch3 == "да") & (ch4 == "да") & (ch5
== "да") & (ch6 == "да") & (ch7 == "да") & (ch8 == "да")){
        //if (true){
            for (int k = 0; k < n; k++){
                writeStream << sbox[k] << " ";
                if ((k+1) % 16 == 0)
                    writeStream << "\n";
            }
            QString iteration = "Итерация: " + QString::number(count_search) +
";\n";

            QString result1 = "1) Отсутствие неподвижных точек: " + ch1 + ";\n";
            QString result2 = "2) Максимальный элемент матрицы частот ниже
порога или равно ему: " + ch2 + "; Значение: " + QString::number(P2) + ";\n";
            QString result3 = "3) Число ненулевых элементов матрицы частот выше
порога или равно ему: " + ch3 + "; Значение: " + QString::number(P3) + ";\n";
            QString result4 = "4) Минимальная степень полинома Жегалкина
координатных функций выше или равна порогу: " + ch4 + "; Значение: " +
QString::number(P4) + ";\n";
            QString result5 = "5) Максимальная разностная характеристика
координатных функций ниже или равна порогу: " + ch5 + "; Значение: " +
QString::number(P5) + ";\n";
            QString result6 = "6) Минимальное значение нелинейности координатных
функций выше или равно порогу: " + ch6 + "; Значение: " + QString::number(P6) +
";\n";
            QString result7 = "7) Глобальный лавинный критерий: " + ch7 + "; Max
sigma: " + QString::number(sigma) + "; Max delta: " + QString::number(delta) +
";\n";
            QString result8 = "8) Минимальная длина циклов и их количество выше
или равно порогу: " + ch8 + "; Длины циклов: " + length_cycle + "; Количество
циклов:" + QString::number(count_cycles) + ";\n";
            writeStream << iteration << result1 << result2 << result3 <<
result4 << result5 << result6 << result7 << result8;
            writeStream << "\n\n";
            count_sbox++; }
        count_search++;
        search_number->setText(search.setNum(count_search));
        find_number->setText(find.setNum(count_sbox));
    }
    file.close();
    if (cancel_model->isDown() == true){
        QString key_string_current;
        for (int i = 0; i < 32; i++){
            key_string_current.append(QString::number(master_key[i]));
            //key_string_current.append(buf.setNum(master_key[i]));
            if (i != 31)
                key_string_current.append(" ");
        }
        QString IV_string_current;
        for (int i = 0; i < 16; i++){
            IV_string_current.append(QString::number(IV_CTR[i]));
            //IV_string_current.append(buf.setNum(IV_CTR[i]));
        }
    }
}

```

```

        if (i != 15)
            IV_string_current.append(" ");
    }
    int poisk = 0;
    for (int i = 0; i < master_key_start_json.size(); i++){
        if ((master_key_string == master_key_start_json[i].toString()) &
            (IV_CTR_string == IV_CTR_start_json[i].toString()) & (n ==
            sizeofbox_json[i].toInt())){
            count_search_json[i] = count_search;
            count_sbox_json[i] = count_sbox;
            master_key_current_json[i] = key_string_current;
            IV_CTR_current_json[i] = IV_string_current;
            poisk = 1;
            break;
        }
    }
    if (poisk == 0){
        master_key_current_json.append(key_string_current);
        IV_CTR_current_json.append(IV_string_current);
        master_key_start_json.append(master_key_string);
        IV_CTR_start_json.append(IV_CTR_string);
        count_search_json.append(count_search);
        count_sbox_json.append(count_sbox);
        sizeofbox_json.append(n);
    }
    QJsonObject myJsonObj;
    myJsonObj["master_key_start"] = master_key_start_json;
    myJsonObj["IV_CTR_start"] = IV_CTR_start_json;
    myJsonObj["count_search"] = count_search_json;
    myJsonObj["count_sbox"] = count_sbox_json;
    myJsonObj["sizeofbox"] = sizeofbox_json;
    myJsonObj["master_key_current"] = master_key_current_json;
    myJsonObj["IV_CTR_current"] = IV_CTR_current_json;
    QJsonDocument doc(myJsonObj);
    json.open(QIODevice::WriteOnly | QIODevice::Text);
    json.write(doc.toJson(QJsonDocument::Indented));
    json.close();
    //вывод сообщения о прерывании поиска
    QString rty;
    QString qwe = "Найдено " + rty.setNum(count_sbox) + " S-блоков.";
    QMessageBox::information(0, "Поиск прерван", qwe);
}
//log_file.close();
//progress_search->hide();
save_box->setDisabled(false);
state_master_key_kuznechik->setReadOnly(false);
state_IV->setReadOnly(false);
size_box->setDisabled(false);
menuBar()->setDisabled(false);
search_box->show();
cancel_model->hide();
search_label->hide();
find_label->hide();
search_number->hide();
find_number->hide();
clear_log->show();
delete [] sbox;
delete [] master_key;
delete [] IV_CTR;
}
void MainWindow::Menu1_Submenu2() {

```

```

QWidget *widget = new QWidget(this);
QGridLayout *layout = new QGridLayout(widget);
open_file = new QPushButton(tr("Открыть файл"));
checkbox = new QPushButton(tr("Начать проверку"));
address_gen = new QLineEdit();
results_check = new QTextBrowser();
address_gen->setReadOnly(true);
setCentralWidget(widget);
layout->addWidget(new QLabel (tr("Проверка предложенного S-блока
(размера 16 или 256) на соответствие требованиям")), 0, 0, 1, 2);
layout->addWidget(address_gen, 1, 0, 1, 1);
layout->addWidget(open_file, 1, 1, 1, 1);
layout->addWidget(checkbox, 1, 2, 1, 1);
layout->addWidget(results_check, 2, 0, 1, 3);
connect(open_file, SIGNAL(clicked(bool)), this,
SLOT(pushButton_clicked()));
connect(checkbox, SIGNAL(clicked(bool)), this,
SLOT(check_box_clicked()));
widget->setAttribute(Qt::WA_DeleteOnClose);
}
void MainWindow::pushButton_clicked()
{
    QString str = QFileDialog::getOpenFileName(0, "Выберите файл, содержащий S-
блок", "", "*.txt");
    address_gen->setText(str); //запись пути файла в строку
    results_check->clear();
}
void MainWindow::check_box_clicked()
{
    if (address_gen->text() == "")
    {
        QMessageBox::warning(0, "Ошибка ввода", "Не указан путь!");
        return;
    }
    QRegExp rx("\\s|\\s"); // регулярное выражение для разбиения
    QStringList sbbox_string;
    //считывание из файла
    QFile file(address_gen->text());
    if (file.open(QIODevice::ReadOnly | QIODevice::Text)){
        while(!file.atEnd()){
            QString str = file.readLine();
            sbbox_string.append(str.split(rx, QString::SkipEmptyParts));
        }
        //вызов функции для проверки s-box
        QString ch1 = "", ch2 = "", ch3 = "", ch4 = "", ch5 = "", ch6 = "", ch7 =
"", ch8 = "", length_cycle = "";
        int P2 = 0, P3 = 0, P4 = 0, P5 = 0, P6 = 0, delta = 0, count_cycles = 0;
        double sigma = 0;
        n = sbbox_string.length();
        if ((n != 16) & (n != 256)){
            QMessageBox::warning(0, "Warning", "Неверно создан файл! Проверьте
формат записи в файле");
            return;
        }
        int *sbox_check = new int [n];
        QString buf_1 = sbbox_string.at(0);
        if (buf_1.length() > 3)
            for (int i = 0; i < n; i++)
                sbox_check[i] = sbbox_string.at(i).toInt(false, 16);
        else

```

```

        for (int i = 0; i < n; i++)
            sbbox_check[i] = sbbox_string.at(i).toInt();
        block_sbbox_mode2(sbbox_check, n, ch1, ch2, ch3, ch4, ch5, ch6, ch7, ch8, P2,
P3, P4, P5, P6, sigma, delta, length_cycle, count_cycles);
        //вывод результата
        results_check->clear();
        results_check->append("Степень S-блока: " + QString::number(n) + ";");
        results_check->append("1) Отсутствие неподвижных точек: " + ch1 + ";");
        results_check->append("2) Максимальный элемент матрицы частот ниже порога
или равно ему: " + ch2 + "; Значение: " + QString::number(P2) + ";");
        results_check->append("3) Число ненулевых элементов матрицы частот выше
порога или равно ему: " + ch3 + "; Значение: " + QString::number(P3) + ";");
        results_check->append("4) Минимальная степень полинома Жегалкина
координатных функций выше или равна порогу: " + ch4 + "; Значение: " +
QString::number(P4) + ";");
        results_check->append("5) Максимальная разностная характеристика
координатных функций ниже или равна порогу: " + ch5 + "; Значение: " +
QString::number(P5) + ";");
        results_check->append("6) Минимальное значение нелинейности координатных
функций выше или равно порогу: " + ch6 + "; Значение: " + QString::number(P6) +
";");
        results_check->append("7) Глобальный лавинный критерий: " + ch7 + "; Max
sigma: " + QString::number(sigma) + "; Max delta: " + QString::number(delta) +
";");
        results_check->append("8) Минимальная длина циклов и их количество выше или
равно порогу: " + ch8 + "; Длины циклов: " + length_cycle + "; Количество
циклов: " + QString::number(count_cycles) + ";");
        delete [] sbbox_check;
    }
void MainWindow::save_box_clicked()
{
    QString fileName = QFileDialog::getSaveFileName(this, tr("Сохранить S-
блоки"), "", tr("*.txt"));
    address_save->setText(fileName); //запись пути файла в строку
}

void MainWindow::define_size(QString state){
    if (state == "16")
        n = 16;
    if (state == "256")
        n = 256;
}
MainWindow::~MainWindow()
{
    delete ui;
}
void MainWindow::closeEvent(QCloseEvent *event)
{
    event->ignore();

    QMessageBox *exitBox = new QMessageBox;
    exitBox->setWindowTitle("Выход");
    exitBox->setText("В случае закрытия программы все несохраненные данные будут
утрачены!\nЗакрыть программу? ");
    exitBox->setStandardButtons(QMessageBox::Yes | QMessageBox::No);
    exitBox->setDefaultButton(QMessageBox::No);
    exitBox->setButtonText(QMessageBox::Yes, "Да");
    exitBox->setButtonText(QMessageBox::No, "Нет");
    int buf = exitBox->exec(); // возврат значения нажатой кнопки
    delete exitBox;
    if (buf == QMessageBox::Yes)

```

```

        event->accept();
    }
}

```

Листинг А.3 – Исходный код файла sbox.cpp

```

#include <sbox.h>
#include "Kuznechik.h"
#include <mainwindow.h>
using namespace std;
void block_sbox_mode2(int *sbox, int n, QString &check1, QString &check2,
    QString &check3, QString &check4, QString &check5, QString &check6, QString
    &check7, QString &check8, int &P2, int &P3, int &P4, int &P5, int &P6, double
    &sigma, int &delta, QString &length_cycle, int &iteration)
{
    int k; //alpha_dec[16];

    if (n == 16)
        k = 4;
    else k = 8;
    int *buf = new int [n];

    int **bin = new int* [n]; // массив для значений функций в 2сс
    for (int i = 0; i < n; i++)
        bin[i] = new int [k];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < k; j++)
            bin[i][j] = 0;
    memcpy(buf, sbox, n*4);
    _dec_bin(buf, bin, n, k); // перевод sbox из 10сс в 2сс

    int **function_table = new int* [k]; // массив для хранения в
    инвертированном виде (строки -> столбцы)
    for (int i = 0; i < k; i++)
        function_table[i] = new int [n];
    for (int i = 0; i < k; i++)
        for (int j = 0; j < n; j++)
            function_table[i][j] = bin[j][i];

    int **matrix_fr = new int* [n-1]; //объявление матрицы частот
    for (int i = 0; i < n-1; i++)
        matrix_fr[i] = new int [n-1];
    matrix_freq(sbox, n, matrix_fr); //заполнение матрицы частот
    proverka1(sbox, check1, n); // проверка на фиксированные точки
    proverka2(matrix_fr, check2, n, P2); // максимум матрицы частот
    proverka3(matrix_fr, check3, n, P3); // число ненулевых элементов
    proverka4(n, check4, P4, function_table); // проверка на степень
    полинома Жегалкина координатных функций
    proverka5(sbox, n, check5, P5); // разностная характеристика
    proverka6(n, check6, P6, function_table); // проверка на нелинейность с
    помощью коэф Уолша-Адамара
    proverka7(n, check7, sigma, delta, function_table); //глобальный
    лавинный критерий
    proverka8(sbox, n, check8, length_cycle, iteration); // проверка на
    циклы

    for (int i = 0; i < n-1; i++)
        delete [] matrix_fr[i];
    delete[] matrix_fr;
    delete [] buf;
    for (int i = 0; i < n; i++)

```

```

        delete [] bin[i];
    delete [] bin;
    for (int i = 0; i < k; i++)
        delete [] function_table[i];
    delete [] function_table;

}

void block_sbox_model(int *sbox, int *master_key, int *IV_CTR, int n, QString
&check1, QString &check2, QString &check3, QString &check4, QString &check5,
QString &check6, QString &check7, QString &check8, int &P2, int &P3, int &P4,
int &P5, int &P6, double &sigma, int &delta, QString &length_cycle, int
&iteration){
    int **matrix_fr = new int* [n-1]; //объявление матрицы частот
        for (int i = 0; i < n-1; i++)
            matrix_fr[i] = new int [n-1];
    for (int k = 0; k < n; k++)
        sbox[k] = 0;
    CTR_crypt(sbox, n, master_key, IV_CTR); // создание sbox при помощи
Кузнечика в режиме гаммирования (берется младший байт)
    int k; //alpha_dec[16];

    if (n == 16)
        k = 4;
    else k = 8;
    int *buf = new int [n];
    int **bin = new int* [n]; // массив для значений функций в 2сс
    for (int i = 0; i < n; i++)
        bin[i] = new int [k];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < k; j++)
            bin[i][j] = 0;

    memcpy(buf, sbox, n*4);
    _dec_bin(buf, bin, n, k); // перевод sbox из 10сс в 2сс
    int **function_table = new int* [k]; // массив для хранения в
инвертированном виде (строки -> столбцы)
    for (int i = 0; i < k; i++)
        function_table[i] = new int [n];
    for (int i = 0; i < k; i++)
        for (int j = 0; j < n; j++)
            function_table[i][j] = bin[j][i];
    proverka1(sbox, check1, n); // проверка на фиксированные точки
    if (check1 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    proverka6(n, check6, P6, function_table); // проверка на нелинейность с
помощью коэф Уолша-Адамара
    if (check6 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];

```

```

        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    proverka4(n, check4, P4, function_table); // проверка на степень
полинома Жегалкина координатных функций
    if (check4 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    proverka5(sbox, n, check5, P5); // дифференциальная равномерность
    if (check5 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    proverka7(n, check7, sigma, delta, function_table); //глобальный
лавинный критерий
    if (check7 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    proverka8(sbox, n, check8, length_cycle, iteration); // проверка на
циклы
    if (check8 == "нет"){

```



```

        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    matrix_freq(sbox, n, matrix_fr); //заполнение матрицы частот
    proverka2(matrix_fr, check2, n, P2); // максимум матрицы частот
    if (check2 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    proverka3(matrix_fr, check3, n, P3); // число ненулевых элементов
    if (check3 == "нет"){
        for (int i = 0; i < n-1; i++)
            delete [] matrix_fr[i];
        delete [] matrix_fr;
        delete [] buf;
        for (int i = 0; i < n; i++)
            delete [] bin[i];
        delete [] bin;

        for (int i = 0; i < k; i++)
            delete [] function_table[i];
        delete [] function_table;
        return;
    }
    for (int i = 0; i < n-1; i++)
        delete [] matrix_fr[i];
    delete [] matrix_fr;
    delete [] buf;
    for (int i = 0; i < n; i++)
        delete [] bin[i];
    delete [] bin;

    for (int i = 0; i < k; i++)
        delete [] function_table[i];
    delete [] function_table;
}

void matrix_freq(int *sbox, int n, int **matrix_fr) {
    int **buf = new int* [n-1]; // матрица разностей
    for (int i = 0; i < n-1; i++)
        buf[i] = new int [n];
}

```

```

        for (int i = 0; i < n-1; i++){
            for (int j = 0; j < n; j++){
                buf[i][j] = sbox[(j+i+1)% n] - sbox[j % n]; //заполнение
матрицы разностей
                if (buf[i][j] < 0)
                    buf[i][j] += n;
            }
        }
        int count = 0;
        for (int k = 0; k < n-1; k++){
            for (int i = 0; i < n-1; i++){
                for (int j = 0; j < n; j++){
                    if (buf[k][j] == i+1)
                        count += 1;
                }
                matrix_fr[k][i] = count; // заполнение матрицы частот
                count = 0;
            }
        }

        for (int i = 0; i < n-1; i++)
            delete [] buf[i];
        delete [] buf;
    }
    //проверка на наличие точек перестановки, которые отображаются сами на себя
    void proverka1(int *sbox, QString &check1, int n){

        check1 = "да";
        for (int i = 0; i < n; i++)
            if (sbox[i] == i){
                check1 = "нет";
                break;
            }
    }
    //проверка условия на максимальный элемент матрицы частот
    void proverka2(int **matrix_fr, QString &check2, int n, int &max){
    //    check2 = "да";
        max = matrix_fr[0][0];
        for (int i = 0; i < n-1; i++){
            for (int j = 0; j < n-1; j++){
                if (matrix_fr[i][j] > max)
                    max = matrix_fr[i][j];
            }
        }
        //проверка условия на максимальный элемент матрицы частот
        if ((n == 16) & (max > 3)) | ((n == 256) & (max > 7))
            check2 = "нет";
        else
            check2 = "да";
    }
    // проверка на число ненулевых элементов в матрице частот
    void proverka3(int **matrix_fr, QString &check6, int n, int &count){
    //    check6 = "да";
        count = 0;
        for (int i = 0; i < n-1; i++){
            for (int j = 0; j < n-1; j++){
                if (matrix_fr[i][j] != 0)
                    count++;
            }
        }
        // проверка на число ненулевых элементов в матрице частот

```

```

        if ((n == 16) & (count < 165)) | ((n == 256) & (count < 41200))
            check6 = "нет";
        else
            check6 = "да";
    }
    void _ascii(char *k, int *a, int m){
        for (int i = 0; i < m; i++){
            a[i] = (char)int(k[i]);
        }
    }
    void _dec_bin( int *buf, int **bin, int n, int k){
        for (int i = 0; i < n; i++){
            for (int j = 0; j < k; j++){
                int c = buf[i] % 2;
                buf[i] /= 2;
                bin[i][k - 1 - j] = c;
            }
        }
    }
    void number_ones(int **buf, int **number, int n){
        for (int i = 0; i < n-1; i++){
            for (int j = 0; j < n; j++){
                int count = 0;
                while (buf[i][j] != 0){
                    if (buf[i][j] % 2 == 1)
                        count++;
                    buf[i][j] /= 2;
                }
                if (count % 2 == 0)
                    number[i][j] = 0;
                else
                    number[i][j] = 1;
            }
        }
    }
    void count_ones_number(int buf, int &number){
        int count = 0;
        while (buf != 0){
            if (buf % 2 == 1)
                count++;
            buf /= 2;
        }
        number = count;
    }
    void proverka4(int n, QString &check8, int &min, int **function_table){
        //проверка на степень полинома Жегалкина
        int k; //alpha_dec[16];

        if (n == 16)
            k = 4;
        else k = 8;
        int *polin = new int [n]; //массив для хранения i-ой строки для метода
треугольника
        for (int i = 0; i < n; i++){
            polin[i] = 0;
        }
        int *anf = new int [k]; // массив для хранения степени полинома Жегалкина
        for (int i = 0; i < k; i++){
            anf[i] = 0;
        }
        int stepen_sbox = 0;
        for (int i = 0; i < k; i++){
            stepen_sbox = 0; // степень полинома sbox
            for (int j = 0; j < n; j++){

```

```

        polin[j] = function_table[i][j];
    }
    treugolnik(polin, n, stepen_sbox);
    anf[i] = stepen_sbox;
}
min = anf[0];
for (int i = 0; i < k; i++)
    if (anf[i] < min)
        min = anf[i];
//проверка на условие
if (((n == 16) & (min < 3)) | ((n == 256) & (min < 7)))// проверка
    check8 = "нет";
else
    check8 = "да";
delete [] polin;
delete [] anf;
}
void treugolnik(int *polinom, int n, int &stepen){ // построение полинома
Жегалкина методом треугольника
    int **metod = new int* [n];
    for (int i = 0; i < n; i++)
        metod[i] = new int [n];
    for (int i = 0; i < n; i++)
        for (int j = 0 ; j < n; j++)
            metod[i][j] = 0;
    for (int j = 0; j < n; j++)
        metod[0][j] = polinom[j];
    int k = n - 1;
    for (int i = 1; i < n; i++){
        for (int j = 0; j < k; j++){
            metod[i][j] = metod[i-1][j] ^ metod[i-1][j+1];
        }
        k--;
    }
    int number_bit = 0;
    for (int i = 0; i < n; i++){
        if (metod[i][0] == 1)
            count_ones_number(i, number_bit);
        if (number_bit > stepen)
            stepen = number_bit;
    }
    for (int i = 0; i < n; i++)
        delete [] metod[i];
    delete [] metod;
}
//проверка на дифференциальную равномерность
void proverka5(int *sbox, int n, QString &check9, int &max){
    int *alpha_dec = new int [n-1]; // коэф _alpha в 10сс
    for (int i = 0; i < n-1; i++)
        alpha_dec[i] = i + 1;
    int *b_dec = new int [n]; // коэф b в 10сс
    for (int i = 0; i < n; i++)
        b_dec[i] = i;
    //массив 255x256 для хранения результатов f(x mod2 a) mod2 f(x)
    int **subtitles = new int* [n-1];
    for (int i = 0; i < n-1; i++)
        subtitles[i] = new int [n];
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n; j++)
            subtitles[i][j] = 0;
    //массив 255x256 для хранения числа совпадений матрицы subtitles с b_dec
    int **count_eval = new int*[n-1];
    for (int i = 0; i < n-1; i++)

```

```

        count_eval[i] = new int [n];

for (int i = 0; i < n-1; i++)
    for (int j = 0; j < n; j++)
        count_eval[i][j] = 0;

for (int i = 0; i < n-1; i++)
    for (int j = 0; j < n; j++)
        subtitles[i][j] = sbox[j ^ alpha_dec[i]] ^ sbox[j];

for (int i = 0; i < n-1; i++){
    for (int m = 0; m < n; m++){
        int count_9 = 0;
        for (int j = 0; j < n; j++){
            if (subtitles[i][j] == b_dec[m])
                count_9++;
        }
        count_eval[i][m] = count_9;
    }
}
max = count_eval[0][0];
for (int i = 0; i < n-1; i++)
    for (int j = 0; j < n; j++)
        if (count_eval[i][j] > max)
            max = count_eval[i][j];
if ((n == 16) & (max <= 4)) | ((n == 256) & (max <= 10))// проверка
    check9 = "да";
else
    check9 = "нет";

for (int i = 0; i < n-1; i++)
    delete [] subtitles[i];
delete [] subtitles;

for (int i = 0; i < n-1; i++)
    delete [] count_eval[i];
delete [] count_eval;
delete [] alpha_dec;
delete [] b_dec;
}
//проверка на нелинейность, используя коэффициент Уолша-Адамара
void proverka6(int n, QString &check10, int &max, int **function_table){
    int k = 0;
    if (n == 16)
        k = 4;
    else k = 8;
    //таблица скалярного произведения (x, y)
    int **multi_x_y = new int* [n];
    for (int i = 0; i < n; i++)
        multi_x_y[i] = new int [n];
    //скалярное произведение векторов y на x
    for (int i = 0; i < n; i++){
        int multi = 0, number_bit = 0;
        for (int j = 0; j < n; j++){
            multi = i & j; // побитовое умножение y(i) на x(j)
            count_ones_number(multi, number_bit); // число единиц для сложения
по mod2
            if (number_bit % 2 == 0)
                multi_x_y[i][j] = 0;
            else
                multi_x_y[i][j] = 1;
        }
    }
}

```

```

    }
}
//определение коэффициентов Уолша-Адамара
int *UA = new int [k];
int max_f = -256, count = 0;
for (int i = 0; i < k; i++){
    for (int j = 0; j < n; j++){
        for (int l = 0; l < n; l++){
            if ((function_table[i][l] ^ multi_x_y[j][l]) == 0)
                count++;
            else
                count--;
        }
        if (max_f < count)
            max_f = count;
        count = 0;
    }
    UA[i] = max_f;
    max_f = -256;
}
//нахождение максимума среди коэф
max = abs(UA[0]);
for (int i = 0; i < k; i++)
    if (max < abs(UA[i])) // модуль в формуле
        max = UA[i];
//значение степени нелинейности
if (n == 256)
    max = 128-max/2; // здесь max - минимальное расстояние до аффинных
функций среди ВСЕХ координатных функций
else if (n == 16)
    max = 8-max/2;
//проверка на условие
if (((n == 256) & (104 <= max) & (max <= 120)) | ((n == 16) & (4 <= max) & (
max <= 6)))
    check10 = "да";
else
    check10 = "нет";

delete [] UA;

for (int i = 0; i < n; i++)
    delete [] multi_x_y[i];
delete [] multi_x_y;}
//глобальный лавинный критерий
void proverka7(int n, QString &check12, double &sigma, int &delta, int
**function_table){
    int k = 0;
    if (n == 16)
        k = 4;
    else k = 8;
    int **autocorr_f = new int* [k];
    for (int i = 0; i < k; i++)
        autocorr_f[i] = new int[n-1];
    //определение значений автокорреляционной функции с значений a, у которых
вес = 1
    int count = 0;
    for (int i = 0; i < k; i++){
        for (int l = 0; l < n-1; l++){
            for (int j = 0; j < n; j++){
                if ((function_table[i][j] ^ function_table[i][(l+1) ^ j]) == 0)
                    count++;
                else

```

```

        count--;
    }
    autocorr_f[i][1] = count;
    count = 0;
}
}
double *sigma_massive = new double [k];
double counting = 0;
sigma = 0;
for (int i = 0; i < k; i++){
    for (int j = 0; j < n-1; j++){
        counting += pow(autocorr_f[i][j], 2);
    }
    sigma_massive[i] = counting;
    counting = 0;
}
for (int i = 0; i < k; i++){
    if (sigma < sigma_massive[i])
        sigma = sigma_massive[i];
}
delta = abs(autocorr_f[0][0]);
for (int i = 0; i < k; i++){
    for (int j = 0; j < n-1; j++){
        if (abs(autocorr_f[i][j]) > delta)
            delta = autocorr_f[i][j];
    }
}
if (((n == 256) & (0 < sigma) & (sigma <= 132096) & (0 <= delta) & (delta <=
56)) | ((n == 16) & (0 < sigma) & (sigma <= 384) & (0 <= delta) & (delta <=
8)))// проверка
    check12 = "да";
else
    check12 = "нет";
delete [] sigma_massive;
for (int i = 0; i < k; i++)
    delete [] autocorr_f[i];
delete [] autocorr_f;}
//проверка на свойства циклов
void proverka8 (int *sbox, int &n, QString &check13, QString &length_cycle, int
&iteration){
    vector<int> cycle;
    vector<int> searchs;
    check13 = "да";
    int x = sbox[0];
    //добавление начальных значений
    cycle.push_back(0);
    searchs.push_back(0);
    int k = 1, start = 0, sum = 0, minim = 0;
    iteration = 0;
    while (sum != n){
        while (x != start){
            //cout << x << " ";
            cycle.push_back(x);
            x = sbox[x];
            k++;
        }
        int search = minim+1;
        int check = 0;
        while (check == 0){
            unsigned int count = 0, i = 0;
            while (i != cycle.size()){
                if (search == cycle[i]){
                    check = 1;
                    search++;
                    count++;
                }
            }
        }
    }
}

```

```

        i = -1;
    }
    if ((i == cycle.size()-1) & (count == 0))
        check = 0;
    i++;
}
if (search < n)
    searchs.push_back(search);
if (check == 0)
    break;
}
cout << "\n";
start = search;
x = sbox[start];
sum +=k;
if (search < n)
    cycle.push_back(search);
length_cycle += QString::number(k) + " ";
iteration++;
if (((n == 256) & ((k < 16) | (iteration > 3))) | ((n == 16) & ((k < 16)
| (iteration > 1)))){
    check13 = "нет";
    //break;
}
k = 1;
if (search >= n)    break;    } }

```

Листинг A.5 – Исходный код файла main.cpp

```

#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.setWindowFlags (Qt::Dialog |Qt::WindowMinimizeButtonHint |
Qt::WindowCloseButtonHint );
    w.show();

    return a.exec();
}

```

Листинг A.6 – Исходный код файла Kuzneshik.h

```

#pragma once
#ifndef __KUZNECHIK_H__
#define __KUZNECHIK_H__
#include <iostream>
#include <windows.h>
void s(int *x, int *pi);
void r(int *x);
void l(int *x);
void x_fun(int *x, int *key);
int LFSR(int *x);
void gen_c(int *c);
void gen_key(int *c, int *k, int *pi);
void cout_c(int *c);
void cout_key(int *k);
void multiple_gf(int *q, int *x);
int razryad(short v);
void network(int *c, int *k, int f, int g, int *pi);
void sdvig(int *x);

```



```
void crypt(int *k, int *a, int *b, int *pi);
void CTR_crypt(int *sbox, int n, int *master_key, int *IV_CTR);
```

```
#endif
```

Листинг А.7 – Исходный код файла mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QMainWindow>
#include <QFormLayout>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QTextBrowser>
#include <QCalendarWidget>
#include <QFileDialog>
#include <QMessageBox>
#include <QComboBox>
#include <QFile>
#include <QProgressBar>
#include <QSpinBox>
#include <QTextStream>
#include <iostream>
#include <QCoreApplication>
#include <QThread>
#include <QPalette>
#include <QDialog>
#include <QJsonArray>
#include <QJsonDocument>
#include <QJsonObject>
#include <QJsonValue>
#include <QJsonParseError>
#include <QCloseEvent>
#include <QTextCodec>
#include <QRegExp>
namespace Ui {
class MainWindow;
}
class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    explicit MainWindow(QWidget *parent = 0);
    ~MainWindow();
    //режим 1 - поиск s-блоков, удовлетворяющих требованиям
    QPushButton *search_box;
    QLineEdit *address_save;
    QPushButton *save_box;
    QLineEdit *state_master_key_kuznechik;
    QLineEdit *state_IV;
    QComboBox *size_box;
    QPushButton *cancel_model;
    QLabel *search_label;
    QLabel *find_label;
    QLabel *search_number;
    QLabel *find_number;
    QPushButton *clear_log;
    //режим 2 - проверка на свойства вводимого s-блока
    QPushButton *open_file;
    QPushButton *checkbox;
    QLineEdit *address_gen;
```

```

        QTextBrowser *results_check;
        int n;
public slots:
        void Menu1_Submenu1();
        void Menu1_Submenu2();
        void pushButton_clicked();
        void check_box_clicked();
        void save_box_clicked();
        void define_size(QString state);
        void search_box_clicked();
        void clear_log_clicked();
private:
        Ui::MainWindow *ui;
        void closeEvent(QCloseEvent *event);
};

```

```

#endif // MAINWINDOW_H

```

Листинг А.8 – Исходный код файла sbox.h

```

#ifndef SBOX_H
#define SBOX_H
#include <fstream>
#include <iostream>
#include <locale>
#include <math.h>
#include <cmath>
#include <ctime>
#include <cstdio>
#include <QString>
void _ascii(char *k, unsigned char *a, int m);
void _dec_bin(int *buf, int **bin, int n, int k);
void matrix_freq(int *sbox, int n, int **matrix_fr);
void number_ones(int **buf, int **number, int n);
void count_ones_number(int buf, int &number);
void treugolnik(int *polinom, int n, int &stepen);
void proverka1(int *sbox, QString &check1, int n);
void proverka2(int **matrix_fr, QString &check2, int n, int &max);
void proverka3(int **matrix_fr, QString &check6, int n, int &count);
void proverka4(int n, QString &check8, int &min, int **function_table);
void proverka5(int *sbox, int n, QString &check9, int &max);
void proverka6(int n, QString &check10, int &max, int **function_table);

void proverka7(int n, QString &check12, double &sigma, int &delta, int
**function_table);
void proverka8 (int *sbox, int &n, QString &check13, QString &length_cycle, int
&iteration);
void block_sbox_model(int *sbox, int *master_key, int *IV_CTR, int n, QString
&check1, QString &check2, QString &check3, QString &check4, QString &check5,
QString &check6, QString &check7, QString &check8, int &P2, int &P3, int &P4,
int &P5, int &P6, double &sigma, int &delta, QString &length_cycle, int
&iteration);
void block_sbox_mode2(int *sbox, int n, QString &check1, QString &check2,
QString &check3, QString &check4, QString &check5, QString &check6, QString
&check7, QString &check8, int &P2, int &P3, int &P4, int &P5, int &P6, double
&sigma, int &delta, QString &length_cycle, int &iteration);
#endif // SBOX_H

```

ПРИЛОЖЕНИЕ Б

(обязательное)

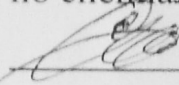
Акт о внедрении результатов выпускной квалификационной работы

УТВЕРЖДАЮ

Заместитель генерального директора

АО «ПНИЭИ»

по специальным вопросам

 /С.А. Сомкин/

«__» _____ 2017 г.

АКТ О ВНЕДРЕНИИ

результатов выпускной квалификационной работы

Старкина А.В.

Комиссия в составе Мигина Н.И. (председатель), Акмаева А.Ж. и Федосеева Д.О. установила, что представленные в выпускной квалификационной работе результаты исследований по определению криптостойкого нелинейного биективного преобразования использованы в тематических исследованиях, выполняемых в ходе опытно-конструкторских работ, проводимых АО «ПНИЭИ».

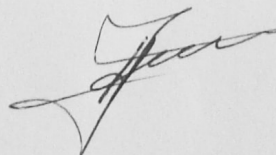
Председатель комиссии

Начальник НИОИ-103

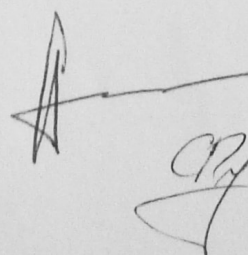
Члены комиссии:

Начальник НИС-103

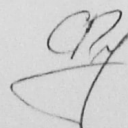
Ведущий инженер-математик



Н.И. Мигин



А.Ж. Акмаев



Д.О. Федосеев